



Universidad
Zaragoza

Trabajo Fin de Grado

Inteligencia Artificial On-Premise para el análisis de
IoCs con datos privados

Autor

Nicolás Pueyo Soria

Directores

Héctor Francia Molinero

Ricardo Julio Rodríguez Fernández

ESCUELA DE INGENIERÍA Y ARQUITECTURA
2026

Agradecimientos

Resumen

Aquí irá el abstract cuando acabe de escribir

Abstract

In English ig

Índice

1. Introducción	1
1.1. Contexto y motivación	1
1.2. Objetivos del proyecto	1
1.3. Estructura del documento	2
2. Conceptos previos	3
2.1. IR e IoCs	3
2.2. Ciberinteligencia de Amenazas (CTI) y OpenCTI	5
2.3. Modelos de Lenguaje de Gran Escala (LLM)	5
2.4. Motores de inferencia local	5
2.5. Generación Aumentada por Recuperación (RAG)	6
3. Análisis, diseño e implementación del sistema	7
3.1. Arquitectura del sistema	10
3.2. Workflow del sistema	11
3.2.1. Consumo y vectorización de OpenCTI	12
3.2.1.1. Ingestión	12
3.2.1.2. Normalización	13
3.2.1.3. Generación de <i>embeddings</i> e inserción en <i>ChromaDB</i>	15
3.2.2. Frontend, backend y motor de RAG	16
3.2.2.1. <i>OpenWebUI</i> y servidor <i>Pipelines</i>	16
3.2.2.2. Backend y motor RAG	19
4. Evaluación y discusión de resultados	25
5. Conclusiones y trabajo futuro	26
Anexos	27
Anexo A. Organización del trabajo.	27
Anexo B. Ficheros desarrollados.	28
Bibliografía	61

Índice de figuras

1.	Diagrama de componentes	11
2.	Diagrama de despliegue con comunicaciones	12
3.	Estructura de directorios del proyecto	17
4.	Workflow completo de secuencia	24
5.	Diagrama de Gantt	27

Índice de cuadros

1.	Resultados para dominio conocido	25
2.	Resultados para dirección IP conocida	25
3.	Resultados para hash conocido	25
4.	Resultados para pregunta por contexto inferido	26
5.	Resultados de relación con actores	26
6.	Resultados de separación fuerte/débil explícita	27
7.	Resultados de pregunta abierta	27
8.	Resultados finales	27
9.	Tabla de tareas y tiempos	28

Índice de listados

1.	Ejemplos de estructuras normalizadas	14
2.	Contextos para 'officeonline.com'	22
3.	client.py	28
4.	ingest.py	29
5.	normalizer.py	31
6.	normalize_all.py	34
7.	embed_all_st.py	36
8.	chroma_load.py	37
9.	docker-compose.yml	39
10.	lily_pipeline.py	40
11.	cybeross_pipeline.py	44
12.	app.py	47
13.	ioc_resolver.py	50
14.	rag_client.py	51
15.	ioc_context_builder.py	52
16.	context_profiles.py	55
17.	builder.py	56

1. Introducción

1.1. Contexto y motivación

En los últimos años, la adopción de tecnologías basadas en Inteligencia Artificial (IA) se ha consolidado como una práctica habitual en el entorno corporativo para optimizar procesos y mejorar la productividad de los empleados. En particular, los Modelos de Lenguaje de Gran Escala (LLM) han demostrado ser herramientas de utilidad para tareas como la síntesis de información y la redacción de informes técnicos. No obstante, este rápido despliegue tecnológico también introduce retos de seguridad significativos. La falta de formación específica en ciberseguridad de muchos usuarios puede dar lugar a malas prácticas en el manejo de la IA, como la exposición accidental de datos corporativos sensibles al enviarlos a APIs comerciales externas.

De forma paralela, la defensa frente a incidentes digitales se apoya cada vez más en la Ciberinteligencia de Amenazas (CTI, por sus siglas en inglés), la cual permite estructurar y correlacionar datos sobre potenciales ataques mediante herramientas de código abierto como OpenCTI. Sin embargo, la gestión diaria de estos repositorios de conocimiento sigue requiriendo una considerable inversión de tiempo por parte de los analistas para procesar e interpretar la información técnica de los nuevos Indicadores de Compromiso (IoC).

En este escenario, surge la oportunidad de integrar la IA generativa como un asistente para facilitar la labor de los analistas en la interpretación y análisis de CTI. Para llevar a cabo esta integración en un contexto empresarial de manera segura, es necesario descartar las soluciones basadas en la nube y optar por modelos locales y privados. De este modo, este proyecto propone el desarrollo de una herramienta desplegada en un servidor privado que, mediante modelos de lenguaje especializados ejecutados localmente, permita a los ingenieros de ciberseguridad consultar, comparar y contrastar la ciberinteligencia de la organización con nuevos indicadores en un tiempo razonable, garantizando la soberanía de los datos sensibles de la empresa.

El presente proyecto se ha desarrollado en el marco de colaboración con la división de Zaragoza de Inetum España S.A., multinacional especializada en consultoría tecnológica y provisión de servicios de TI. En Aragón, Inetum desempeña un papel relevante al gestionar las infraestructuras y los servicios informáticos de entidades públicas de gran envergadura, como la Diputación General de Aragón (DGA) y el Servicio de Salud Aragonés (SALUD). En este contexto empresarial, la infraestructura asignada para el despliegue de la solución consta de entornos de virtualización Linux (RedHat 10), sin entorno de escritorio, sobre hardware de centro de datos basados únicamente en capacidad de procesamiento (CPU) y memoria RAM, prescindiendo por completo de aceleración gráfica por GPU. Además, con el fin de garantizar la soberanía, la transparencia y la viabilidad de la arquitectura propuesta, el sistema se implementará de manera íntegra utilizando herramientas y soluciones de software de código abierto.

1.2. Objetivos del proyecto

El objetivo general de este proyecto es diseñar, implementar y evaluar una plataforma privada y local de asistencia conversacional orientada a la ingeniería de ciberseguridad, integrada con una plataforma de gestión de ciberinteligencia de amenazas (CTI). El

sistema propuesto debe permitir la consulta interactiva y el análisis contextualizado de Indicadores de Compromiso (IoC) garantizando la soberanía de los datos sensibles y optimizando el rendimiento de modelos de lenguaje bajo restricciones estrictas de hardware basadas exclusivamente en procesamiento por CPU y memoria RAM.

Para alcanzar este fin, se proponen los siguientes objetivos específicos:

- **Despliegue y configuración de un entorno local aislado:** Implementar una infraestructura privada que albergue un entorno de ejecución para modelos de lenguaje y una interfaz gráfica de usuario accesible para los analistas, operando de manera local y sin dependencias de servicios externos en la nube.
- **Desarrollo de un pipeline de recuperación semántica:** Diseñar un mecanismo de indexación, almacenamiento vectorial y recuperación semántica de datos que permita alimentar las consultas del asistente conversacional con la ciberinteligencia y el contexto histórico de la organización de forma precisa.
- **Integración y consulta bidireccional con sistemas de ciberinteligencia:** Establecer una conexión con la API de la plataforma de ciberinteligencia del cliente para permitir que el asistente interactúe con el grafo de conocimiento existente, traduciendo consultas en lenguaje natural a búsquedas estructuradas de observables y amenazas.
- **Evaluación comparativa de modelos de lenguaje especializados:** Evaluar el rendimiento, el nivel de detalle técnico, la tasa de alucinaciones y la precisión de múltiples modelos de lenguaje de diferentes escalas de parámetros en tareas complejas de correlación y análisis de indicadores.

1.3. Estructura del documento

El presente documento se estructura en diferentes secciones con subsecciones. En la segunda sección se explican los conocimientos teóricos previos a tener en cuenta sobre diferentes conceptos clave para entender el desarrollo del proyecto. En la tercera sección se detalla cuál es la arquitectura del proyecto, qué componentes se han utilizado, qué tecnologías se han usado para desarrollar o desplegar cada uno de estos componentes, así como qué organización tienen y cómo se despliegan físicamente cada uno, y finalmente explica el workflow general del sistema, es decir, cómo cumple cada componente su cometido. En la cuarta sección se detallan las pruebas que se han realizado al sistema, se recogen y presentan los resultados obtenidos y se realiza un análisis de los mismos. Finalmente, en la última sección, la quinta, se analiza todo el contexto y trabajo desarrollado para extraer unas conclusiones, y se reflexiona sobre qué otras funcionalidades se podrían añadir al proyecto en caso de continuar con el desarrollo.

2. Conceptos previos

2.1. IR e IoCs

En el ámbito de la ciberseguridad corporativa, la Respuesta a Incidentes (IR, por sus siglas en inglés de *Incident Response*) constituye la metodología estructurada y sistemática que adopta una organización para la detección, contención, mitigación y recuperación frente a incidentes de seguridad y ciberataques. De acuerdo con los estándares de la industria, el objetivo primordial de la respuesta a incidentes abarca tanto la prevención proactiva de ciberataques antes de que estos lleguen a materializarse, como la minimización del impacto financiero, el daño reputacional y la interrupción de los procesos de negocio en caso de que una intrusión se complete de manera exitosa.

Para operativizar esta estrategia, las organizaciones definen sus tecnologías, responsabilidades y procedimientos dentro de un Plan de Respuesta a Incidentes (IRP, *Incident Response Plan*). Este plan actúa como el componente técnico de la gestión integral de incidentes, especificando cómo deben identificarse, aislarse y resolverse los diferentes vectores de ataque de forma coordinada. En la actualidad, la integración de soluciones de ciberseguridad asistidas por inteligencia artificial permite acelerar este proceso drásticamente, ayudando a los analistas a predecir posibles canales de ataque, automatizar la clasificación de alertas y correlacionar comportamientos sospechosos antes de que escalen a incidentes críticos.

Para el estudio del estado de la ciberinteligencia en el marco de la respuesta a incidentes, los analistas dependen de los Indicadores de Compromiso (IoCs, *Indicators of Compromise*). En el campo de la informática forense y el análisis de malware, un IoC se define como cualquier artefacto, registro o evidencia digital identificado en un sistema operativo, un dispositivo de red o un flujo de tráfico que permite constatar, con un determinado nivel de confianza, la presencia de una actividad maliciosa o el compromiso de un activo.

En el marco del estándar internacional STIX 2.1, la arquitectura de datos sobre la que se fundamenta la plataforma OpenCTI utilizada en este proyecto, es imperativo establecer una distinción técnica entre un *Observable* (SCO, *STIX Cyber Observable*) y un *Indicador* (SDO, *STIX Domain Object*):

- **Observable (SCO):** Representa un dato técnico en bruto desprovisto de intencionalidad o valoración ética. Una dirección IP, un dominio o el hash de un binario son observables que pueden pertenecer tanto a servicios legítimos de la infraestructura como a servidores del atacante.
- **Indicador (SDO):** Es la entidad que aporta contexto semántico y de seguridad al observable. Un indicador asocia un patrón de búsqueda (definido bajo estándares como STIX Pattern, reglas YARA o firmas Sigma) a un observable específico, determinando un período de validez temporal, relaciones con campañas de actores de amenazas y un nivel de certeza técnica de maliciosidad.

Para el alcance del sistema desarrollado en este proyecto, se considerarán y analizarán los siguientes tipos de observables e indicadores de compromiso técnicos:

- **URLs** identificadas como maliciosas, habitualmente vinculadas a servidores de descarga de cargas útiles (*payloads*), portales de suplantación de identidad

(*phishing*) o paneles de control de botnets.

- **Hashes** de ficheros identificados como códigos dañinos. El uso de funciones de dispersión criptográfica permite identificar unívocamente muestras de malware sin necesidad de procesar el archivo completo. Para este proyecto se contemplan:
 - **SHA256** (64 caracteres hexadecimales), estándar actual por su resistencia a colisiones.
 - **SHA1** (40 caracteres hexadecimales), de uso remanente en bases de datos históricas.
 - **MD5** (32 caracteres hexadecimales), obsoleto criptográficamente pero común en la categorización rápida de amenazas.
- **Direcciones IPv4** identificadas como maliciosas, asociadas comúnmente a infraestructura de Comando y Control (C2), nodos de salida Tor o proxies utilizados para el escaneo de vulnerabilidades.
- **Dominios** web identificados como maliciosos, utilizados para enmascarar direcciones IP de infraestructura atacante mediante técnicas de resolución dinámica de nombres.

A nivel estratégico, la utilidad de estos IoCs se rige bajo la *Pirámide del Dolor* de David Bianco[1]. Este modelo conceptual organiza los indicadores de amenazas según el grado de dificultad (o "dolor") que causa al atacante el hecho de que un defensor los detecte y bloquee:

1. **Base (Hashes, IPs y Dominios):** Constituyen el nivel más bajo. Son extremadamente sencillos y baratos de modificar para el adversario. Un atacante puede alterar un solo byte de su código para cambiar el hash criptográfico del malware, o rotar de dirección IP en segundos mediante servicios de red dinámica, eludiendo bloqueos automatizados estáticos.
2. **Medio (Artefactos de red y de host):** Requiere que el atacante modifique los patrones de comunicación de sus herramientas o las trazas que estas dejan en los registros de los sistemas comprometidos.
3. **Cúspide (Herramientas y TTPs):** Las Tácticas, Técnicas y Procedimientos (TTPs) definen el comportamiento operativo real y los hábitos del atacante. Obligar a un atacante a cambiar sus TTPs es el logro de mayor valor defensivo, ya que le exige reestructurar por completo su metodología y reaprender tácticas de infiltración.

La integración de la Inteligencia Artificial propuesta en este trabajo encuentra su motivación en esta jerarquía. Mientras que los sistemas automáticos de seguridad tradicionales (como cortafuegos o antivirus) son excelentes procesando de manera rígida los observables de la base (hashes e IPs), carecen de la capacidad cognitiva para relacionar estos datos con comportamientos complejos. La incorporación de un modelo de lenguaje de gran escala (LLM) permite a los ingenieros de ciberseguridad procesar el texto no estructurado y los grafos de conocimiento de OpenCTI para correlacionar observables simples con el comportamiento general de la amenaza en la cúspide de la pirámide, forzando un impacto real sobre el coste operativo del atacante.

2.2. Ciberinteligencia de Amenazas (CTI) y OpenCTI

La Ciberinteligencia de Amenazas (CTI, por sus siglas en inglés de *Cyber Threat Intelligence*) consiste en la recopilación, análisis y estructuración de información sobre amenazas activas o potenciales con el fin de anticipar, detectar y mitigar ataques de forma proactiva. Esta disciplina permite a los equipos de seguridad comprender el contexto, las motivaciones y las metodologías de los adversarios, optimizando la toma de decisiones estratégicas y los tiempos de respuesta ante incidentes.

Para centralizar y procesar este conocimiento, las organizaciones emplean plataformas especializadas (TIP, *Threat Intelligence Platforms*) como OpenCTI, una herramienta de código abierto diseñada para estructurar, almacenar, organizar y visualizar información técnica y no técnica sobre ciberamenazas. El núcleo de OpenCTI se fundamenta en un grafo de conocimiento basado en el estándar internacional STIX 2.1. En esta arquitectura, la inteligencia se modela mediante dos elementos[2]:

- **Nodos (Entidades):** Representan conceptos de alto nivel, conocidos como objetos de dominio o *STIX Domain Objects* (SDO), tales como Malware, Actores de Amenazas o Vulnerabilidades.
- **Aristas (Relaciones):** Modelan las conexiones lógicas entre las entidades y se asocian a observables técnicos o *STIX Cyber Observables* (SCO), como direcciones IP, dominios o hashes de archivos, que sirven de evidencia de la actividad.

2.3. Modelos de Lenguaje de Gran Escala (LLM)

Los Modelos de Lenguaje de Gran Escala (LLM, por sus siglas en inglés) son sistemas basados en redes neuronales profundas diseñados para comprender, procesar y generar lenguaje natural. Formalmente, un LLM se define como una función matemática compleja cuya entrada y salida consisten en listas de números enteros[3]. Para que el modelo pueda procesar el texto en lenguaje humano, este debe someterse a un proceso de tokenización, en el cual se divide el flujo de caracteres en unidades discretas menores llamadas tokens (que pueden ser palabras, subpalabras o caracteres individuales)[4]. Este proceso mapea el texto a valores enteros dentro de un rango determinado por el tamaño de su vocabulario $[0, V - 1]$, donde V representa la dimensión del vocabulario del modelo[3].

En el contexto corporativo, la adopción de estos modelos se divide principalmente entre servicios de infraestructura gestionada en la nube y despliegues locales. Si bien el uso de APIs comerciales en plataformas en la nube (como las soluciones de infraestructura de AWS) proporciona una gran capacidad de cómputo y escalabilidad, la transferencia de datos de seguridad hacia servidores externos introduce riesgos severos de exposición, fugas de información y fallas de cumplimiento normativo. Por esta razón, el estado del arte en ciberseguridad se orienta firmemente hacia el despliegue local de modelos de pesos abiertos de menor tamaño, garantizando la soberanía absoluta de los datos sensibles de la organización[5].

2.4. Motores de inferencia local

Mientras que un LLM representa la estructura estática de la red neuronal y sus pesos matemáticos almacenados en disco, el motor de inferencia es el software de ejecución

(*runtime*) activo que carga dichos pesos en memoria y realiza las operaciones algebraicas necesarias para responder a las consultas del usuario. En un servidor privado que carece de aceleradores gráficos (GPU), el motor de inferencia debe ejecutar el modelo utilizando exclusivamente el procesador (vCPU en contextos de centros de datos) y la memoria RAM del sistema.

En este escenario de inferencia puramente basado en CPU, la velocidad del sistema se ve fuertemente limitada. El cuello de botella principal no radica en la capacidad de cómputo de la vCPU, sino en el ancho de banda físico de la memoria RAM, puesto que el procesador debe leer la totalidad de los parámetros del modelo para generar cada token individual. Mientras que las GPUs utilizan memoria especializada de gran velocidad con un ancho de banda de entre 600 y 1500 GB/s, la memoria RAM convencional de un servidor host suele transferir datos a velocidades de entre 50 y 200 GB/s.

Esta limitación física provoca que la asignación de hilos de ejecución en el motor de inferencia no escale de manera lineal con el número de núcleos virtuales (vCPUs) asignados. Si se configuran demasiados hilos de ejecución concurrentes en el motor de inferencia para procesar un modelo pesado, los hilos terminarán compitiendo por los mismos canales físicos de acceso a la memoria RAM. Esta contención de memoria genera un cuello de botella técnico que degrada drásticamente el rendimiento del sistema y reduce la tasa de generación de texto (tokens por segundo), por lo que la optimización de estos sistemas requiere limitar de forma explícita el número de hilos de ejecución a un valor cercano al número de núcleos físicos del procesador.

2.5. Generación Aumentada por Recuperación (RAG)

La Generación Aumentada por Recuperación (RAG, por sus siglas en inglés de *Retrieval-Augmented Generation*) es una técnica que optimiza el comportamiento de un modelo de lenguaje al complementar sus instrucciones (*prompts*) de entrada con información externa relevante sin necesidad de modificar físicamente los parámetros del modelo[4]. El flujo operativo de RAG consta de tres etapas lógicas:

1. **Recuperación:** Ante una consulta del analista, el sistema busca de manera automática los fragmentos de documentos o informes técnicos más relevantes dentro de una base de datos.
2. **Aumentación:** El sistema añade estos fragmentos recuperados al *prompt* original del usuario en forma de contexto informativo verídico.
3. **Generación:** El LLM procesa la consulta enriquecida y genera una respuesta redactada en lenguaje natural que se fundamenta estrictamente en la evidencia proporcionada, reduciendo de manera drástica las alucinaciones del sistema[4].

Para dar soporte y agilizar la etapa de recuperación en tiempo real, se emplean las bases de datos vectoriales (como ChromaDB o Qdrant), las cuales almacenan los textos de ciberinteligencia previamente convertidos en vectores numéricos o *embeddings* que capturan su significado semántico. De este modo, la base de datos es capaz de localizar de forma matemática los fragmentos de información más pertinentes según la intención de la consulta del analista, inyectándolos instantáneamente en el contexto de la IA[3].

3. Análisis, diseño e implementación del sistema

Tomando como punto de partida la infraestructura tecnológica preexistente de la organización, la cual dispone de una instancia de OpenCTI desplegada de manera independiente en un nodo dedicado de la red corporativa, el desarrollo práctico de este trabajo se centra en diseñar e implementar una pasarela de comunicación bidireccional y segura con dicha base de conocimiento.

Con el propósito de estructurar de forma modular el asistente inteligente, cuya finalidad es procesar y correlacionar información de seguridad dentro del perímetro de la empresa, la arquitectura del sistema propuesto se articula en torno a los siguientes componentes:

- **Motor de inferencia local y selección de modelos:** Para la ejecución de los modelos de lenguaje de gran escala bajo un entorno local, se evaluaron diversas alternativas de motores de inferencia, entre las que destacaron Ollama y vLLM. Si bien vLLM ofrece un rendimiento excepcional en la gestión de solicitudes concurrentes gracias al algoritmo de optimización de memoria *PagedAttention* [6], su arquitectura está diseñada de manera prioritaria para operar bajo aceleración por hardware gráfico (GPU). Debido a la restricción de infraestructura impuesta en este proyecto, consistente en operar exclusivamente sobre procesamiento por CPU, se seleccionó Ollama como el motor de inferencia local principal. Ollama, desarrollado sobre la biblioteca nativa en C/C++ llama.cpp, proporciona una capa de abstracción eficiente para entornos virtualizados, facilidad de configuración multimodelo mediante archivos de instrucción (*Modelfiles*) y una API local compatible con los estándares de la industria.

Para posibilitar la interacción con el motor, los modelos de lenguaje se cargan en su versión cuantizada utilizando archivos de extensión `.gguf`. En cuanto a los modelos de lenguaje seleccionados, se descartaron las opciones generalistas en favor de modelos especializados en ciberseguridad mediante ajuste fino (*fine-tuning*). Esta especialización garantiza que la IA sea capaz de interpretar terminología técnica de ciberinteligencia, correlacionar observables y procesar de manera segura datos potencialmente ofensivos (como volcados de registros o firmas de malware) sin sufrir de bloqueos por políticas de seguridad corporativas comerciales. Con el fin de realizar una evaluación comparativa bajo los recursos asignados al servidor, se seleccionaron dos modelos especializados de escalas de parámetros diferenciadas:

- **Lily-Cybersecurity-7B-v0.2:** Modelo ligero desarrollado por Sego Lily Labs, ajustado sobre la arquitectura de Mistral-7B-Instruct-v0.2 a partir de un conjunto de 22,000 instrucciones de seguridad manuales [7]. En su cuantización a 4 bits, presenta un peso de archivo de aproximadamente 4.37 GB, lo que permite una inferencia ágil y fluida con un consumo mínimo de memoria RAM.
- **CyberOSS-2.0-20B (CyberPal-2.0-20B):** Modelo avanzado de la suite de seguridad de código abierto CyberPal 2.0, ajustado sobre la arquitectura gpt-oss-20b mediante el pipeline de enriquecimiento y trazado de razonamiento técnico *SecKnowledge 2.0* [8]. En su cuantización de 4 bits (Q4_K_M), el tamaño del archivo asciende a 15.8 GB, demandando un mayor esfuerzo de lectura por parte de la memoria RAM del servidor, pero aportando un análisis conceptual de nivel experto.

La elección de esta pareja de modelos permite someter la herramienta a una validación práctica frente a los benchmarks de referencia de la literatura científica. De acuerdo con el estudio de evaluación integral publicado en "Toward Cybersecurity-Expert Small Language Models" (arXiv:2510.14113) [8], Lily-Cybersecurity-7B-v0.2 obtiene una puntuación promedio de 53.38 % en la resolución de tareas complejas de seguridad y ciberinteligencia, mientras que CyberPal-2.0-20B alcanza una puntuación de 80.33 % en el mismo conjunto de pruebas (CTI-Bench), superando incluso a modelos comerciales cerrados en la correlación de vulnerabilidades e identificación de tácticas.

- **Interfaz gráfica de usuario y orquestación de flujos (Open WebUI):** Para proporcionar a los analistas de ciberseguridad un entorno visual, accesible e interactivo, se evaluaron diversas plataformas autohospedadas capaces de desplegarse mediante contenedores Docker y configurarse de manera sencilla a través de parámetros de entorno. En una primera fase de diseño, se contempló la implementación de la interfaz *ChatUI* desarrollada por Hugging Face. Sin embargo, este entorno fue descartado debido a incompatibilidades severas en su integración nativa con el motor de inferencia local Ollama. Durante las pruebas de concepto, la ambigüedad en la documentación oficial respecto a su interconexión local propició fallos críticos en el control del estado, tales como la imposibilidad de mapear y listar dinámicamente los modelos de Ollama, y respuestas de los modelos no mostradas en la interfaz.

Para solucionar estos inconvenientes, se seleccionó la plataforma *Open WebUI*. Esta herramienta destaca por su compatibilidad nativa e inmediata con la API de Ollama, configuración de parámetros tanto de forma declarativa (en la configuración de Docker) como en caliente desde su panel de administración web [9]. Un factor tecnológico determinante para su elección fue la disponibilidad de su arquitectura desacoplada de flujos de trabajo, denominada *Pipelines*. Este servidor satélite programado en Python actúa como un *middleware* capaz de interceptar, filtrar y modificar las peticiones dirigidas al modelo de lenguaje [10]. Esta capacidad resulta crítica para el alcance de este proyecto, ya que proporciona el punto de anclaje necesario para inyectar lógica personalizada, realizar llamadas a servicios externos y canalizar las búsquedas de ciberinteligencia dentro de la base de datos vectorial de forma transparente para el analista, facilitando así la integración del sistema RAG.

- **Backend, motor de RAG y pipelines:** Este bloque representa el núcleo diferencial y el valor de desarrollo propio de este proyecto. Mientras que los componentes analizados hasta ahora se basan en el despliegue, parametrización y organización arquitectónica de imágenes de Docker (como Open WebUI o la base de datos vectorial), es en esta capa donde se introduce la lógica personalizada que gobierna el flujo, tratamiento y enriquecimiento de los datos de seguridad. Aunque se describen como componentes diferenciados, el servidor de *Pipelines*, el backend de servicios y el motor de RAG operan como un único subsistema lógicamente acoplado debido a su constante cooperación en tiempo real.

El flujo de procesamiento de la información se inicia cuando el analista realiza una consulta en la interfaz gráfica que menciona un indicador de compromiso (IoC), momento en el cual la solicitud es interceptada de manera inmediata por

el servidor de *middleware Pipelines*. En lugar de transmitir la petición de forma directa al motor de inferencia local, el *pipeline* extrae el *payload* y desvía la solicitud hacia un *endpoint* específico de nuestro backend. Al recibir este paquete, el backend analiza la petición y llama a las funciones del motor de RAG, el cual recupera contexto técnico sobre el indicador consultando de manera estructurada tanto la API de OpenCTI como la base de datos vectorial local (ChromaDB). Una vez obtenido este contexto histórico y relacional, se devuelve en forma de respuesta al *pipeline*, encargado de reformular el *prompt* del usuario inyectando la información recuperada y de transmitir la instrucción final enriquecida al modelo correspondiente para que este realice su razonamiento.

Para la construcción del backend de servicios se seleccionó el marco de trabajo (*framework*) FastAPI. Esta elección se fundamenta principalmente en su sencillez y en que permite un desarrollo de *endpoints* de manera rápida y eficiente, facilitando un acoplamiento ágil con el resto de componentes escritos en Python. Asimismo, FastAPI destaca por su capacidad para gestionar de manera óptima la asincronía de forma nativa a través del servidor web de aplicaciones de bajo nivel Uvicorn, encargado de procesar la especificación ASGI de manera eficiente. Como un añadido a esta infraestructura, FastAPI utiliza internamente Starlette para dar soporte a la asincronía y se integra de forma directa con Pydantic para la validación declarativa de datos mediante anotaciones de tipo, lo que garantiza la integridad de los *payloads* intercambiados sin penalizar la velocidad de desarrollo.

- **Motor de base de datos vectorial (ChromaDB):** La incorporación de este componente responde a la necesidad de complementar la información relacional y estructurada obtenida de la API de OpenCTI con capacidades de recuperación semántica profunda. Mediante el uso de un modelo local de generación de vectores (*embeddings*), el sistema procesa, fragmenta e indexa tanto los informes técnicos de amenazas como los metadatos de los observables extraídos de la plataforma de ciberinteligencia. De este modo, ante una consulta del analista, la base de datos permite ejecutar búsquedas de similitud para deducir inferencias latentes y proporcionar un contexto complementario de alto valor que no se encuentra explícitamente enlazado en el grafo de conocimiento original.

Para la implementación de este almacén de vectores se seleccionó *ChromaDB*. En primer lugar, se define como una base de datos vectorial de código abierto altamente ligera y optimizada para despliegues locales (*on-premise*), lo que evita introducir una sobrecarga de infraestructura compleja o costosa en el servidor. En segundo lugar, su arquitectura modular permite que la base de datos se ejecute de forma embebida directamente dentro del proceso de Python o de manera independiente mediante un contenedor, lo que optimiza drásticamente el uso de recursos y memoria RAM en entornos limitados a CPU. Finalmente, destaca su excelente compatibilidad con el ecosistema de desarrollo de Python y su amplia adopción en la comunidad, lo que garantiza una API sencilla para operaciones de inserción, consulta, actualización y borrado (CRUD), así como un soporte nativo para el filtrado avanzado de metadatos.

- **Módulo de comunicación, ingesta y normalización de ciberinteligencia:** Este componente representa la capa de software desarrollada a medida para interactuar con la plataforma OpenCTI, resolviendo la adquisición de datos de

seguridad y su transformación hacia la base de datos vectorial. La conectividad lógica con el servidor se implementa bajo el patrón de diseño *Singleton* con soporte multihilo (*thread-safe*) mediante un mecanismo de exclusión mutua (*lock*), garantizando que exista una única instancia activa del cliente de API oficial (`OpenCTIApiClient`) durante la ejecución del sistema, lo que evita la saturación de conexiones GraphQL en el nodo de OpenCTI.

Sobre esta infraestructura de conexión se articulan tres subcomponentes de desarrollo propio en Python:

- **Cliente de consultas semánticas (`OpenCTIRAGClient`):** Componente que se ejecuta en tiempo real dentro del flujo de la conversación. Permite realizar búsquedas exactas e intermedias sobre observables e indicadores bajo la estructura del estándar STIX 2.1, rastreando de forma automática las relaciones lógicas (*relationships*) creadas por los analistas y localizando reportes técnicos relevantes que sirvan como evidencia contextualizada para el modelo.
- **Ingestor de entidades (`OpenCTIIngestor`):** Módulo encargado del aprovisionamiento masivo de datos históricos. Implementa el protocolo de paginación oficial basado en cursores de OpenCTI (`withPagination=True` y `endCursor`) para descargar de forma secuencial y ordenada colecciones completas de actores de amenazas, malware, tácticas, reportes e indicadores técnicos, almacenando la información de ciberinteligencia en bruto localmente en formato JSON.
- **Normalizador semántico e indexador (`OpenCTINormalizer`):** Clase de vital importancia para el RAG que resuelve el grafo de relaciones de OpenCTI (como la arista *uses* que vincula a un actor de amenazas con un malware específico o un TTP de MITRE ATT&CK). El normalizador extrae las relaciones complejas y las unifica en plantillas textuales estructuradas y metadatos legibles para la IA. Posteriormente, un script de vectorización automatizado codifica estos textos normalizados en representaciones densas de 384 dimensiones empleando el modelo local `all-MiniLM-L6-v2`, insertándolos finalmente mediante un algoritmo por lotes (*batches*) en ChromaDB bajo una métrica de similitud de distancia del coseno para dar soporte a las búsquedas de proximidad semántica.

3.1. Arquitectura del sistema

La arquitectura de la solución propuesta se ha diseñado siguiendo un paradigma modular y desacoplado. Por restricciones corporativas, sólo se dispone de una máquina virtual, por lo que todo está desplegado en la misma. No obstante, el sistema está diseñado para ser fácilmente distribuido. Esta topología, detallada en el diagrama de componentes de la Figura 1, permite separar de forma estricta el plano de interacción con el usuario e inferencia local del plano de ingesta, normalización y ciberinteligencia de amenazas.

A nivel de infraestructura, la comunicación entre los componentes se realiza a través de puertos y endpoints específicos dentro de una red virtual interna, articulándose en dos bloques tecnológicos principales:

- **Dominio de Interacción e Inferencia Local (Plano Izquierdo):** Este dominio gestiona la interfaz del analista y la ejecución de la IA. El componente *OpenWebUI* actúa como el frontend del sistema, comunicándose directamente con el puerto 9099 del *Pipeline Server*. Este último actúa como un *middleware* de intercepción y enrutamiento, derivando las consultas de enriquecimiento hacia el *Backend RAG* en el puerto 9000, concretamente al endpoint `/query-context` y transmitiendo el prompt final aumentado hacia el endpoint `/api/chat` de *Ollama* en el puerto 11434, el cual expone localmente los modelos *Lily* y *CyberOSS*.
- **Dominio de Ciberinteligencia, Ingesta y Vectorización (Plano Derecho):** Este dominio administra la adquisición de conocimiento de seguridad. El componente *OpenCTI Client* encapsula la conexión única hacia el puerto 8080 de la instancia externa de *OpenCTI*. De este cliente dependen de manera exclusiva el *Ingestor* (responsable del aprovisionamiento masivo de SDOs y SCOs) y el *Backend RAG* (responsable de las consultas semánticas y relacionales en caliente). Ambos módulos escriben y consultan, respectivamente, la base de datos vectorial *ChromaDB* a través de su API REST en el puerto 8080.

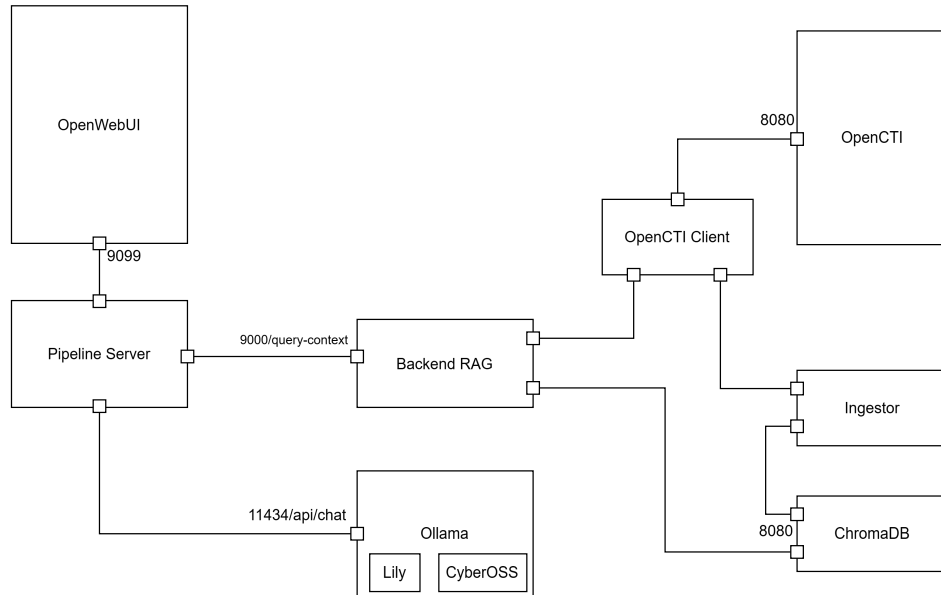


Figura 1: Diagrama de componentes

3.2. Workflow del sistema

En la Figura 2 se detalla concretamente la organización del despliegue de todos los artefactos mencionados, así como las comunicaciones que tienen entre ellos. Las comunicaciones se expresan como flechas etiquetadas. Una flecha bidireccional ($\longleftrightarrow$) significa que se manda una *request* y se espera *response* (por ejemplo, la flecha etiquetada con 1 entre *OpenWebUI* y *Pipelines*, mientras que una flecha en una sola dirección ($\longrightarrow$) significa que simplemente se envían datos en una sola dirección (esto solo se da entre el *ingestor* y *ChromaDB*, etiquetada con II).

Una vez definida la distribución lógica y física de la infraestructura, es fundamental definir la dinámica temporal que sigue el sistema. Como se ha mencionado previamente, el sistema sigue dos flujos de datos diferentes, por una parte está la ingesta, vectorización

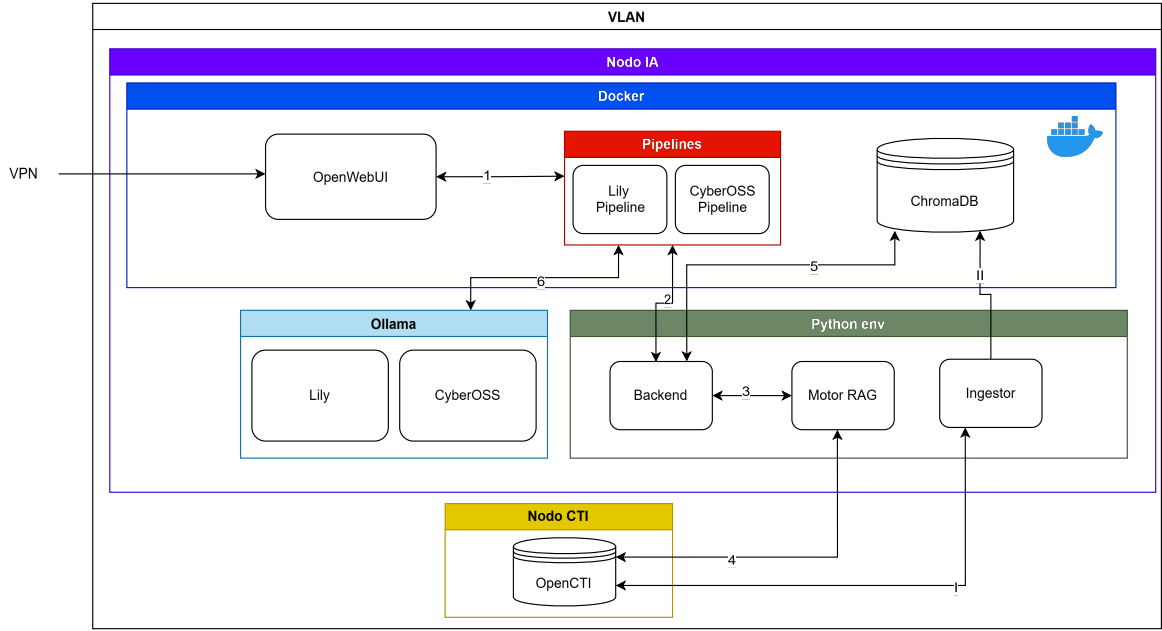


Figura 2: Diagrama de despliegue con comunicaciones

y almacenamiento de datos de OpenCTI (módulo *Ingestor* que interactúa con *OpenCTI* y *ChromaDB*), cuyas comunicaciones están etiquetadas con números romanos en el la Figura 2 (I y II).

Finalmente, en la Figura 3 se detalla dentro del propio **Nodo IA**, la distribución de los directorios y los ficheros desarrollados para llevar a cabo el proyecto, cuyo contenido se puede ver además en el Anexo B: Ficheros desarrollados. Como se observa en la disposición jerárquica de la ilustración, se separan las funciones, por ejemplo, los despliegues de *Docker* se centralizan en un fichero `docker-compose.yml` en el directorio `chat-feats`, los modelos de IA para definir en Ollama se guardan en el directorio `modelos_ia`, los *pipelines* tienen su propio directorio `openwebui_pipelines`, sincronizado con el directorio de *pipelines* de dentro del contenedor, y todos los scripts de *Python* se alojan en el directorio `python-scripts`, separando el módulo de ingestión de datos en `opencti`, el motor de RAG en `rag`, y finalmente, el programa principal que pone en marcha el motor, `main.py`.

3.2.1. Consumo y vectorización de OpenCTI

3.2.1.1. Ingestión

El cometido de este módulo es conectarse a OpenCTI mediante el SDK (provisto por el paquete oficial *pycti*), configurando los credenciales adecuados (*IP* de *OpenCTI* y clave de la API). El cliente es pesado, y para evitar ser cargado múltiples veces, por ejemplo, por diferentes hilos de ejecución, ralentizando el sistema completo, sigue un patrón *Singleton*.

Para cada tipo de entidad CTI que se cargará (concretamente *threat actors*, *malware*, *attack patterns*, *indicators* y *reports*), el módulo define qué atributos necesita recuperar. Para todos los tipos de objeto se cargan identificador, nombre, descripción y fecha de

creación. Para *Threat Actors* y *Reports*, estos son los únicos campos recuperados. Se recuperan campos adicionales para:

- **Malware:** Para este grupo se recupera también el campo `malware_types`, que define a qué categorías de malware pertenece (e.g. *ransomware*, *trojan*, *backdoor*)
- **Attack Patterns:** Para este grupo se recupera además el campo `x_mitre_id`, que es el identificador MITRE asociado a una técnica cuando el *Attack Pattern* pertenece al modelo MITRE ATT&CK[11]
- **Indicators:** Se cargan adicionalmente los campos `pattern_type` y `pattern`, que indican la sintaxis en la que está escrito el campo `pattern`, y la expresión técnica que define el *IoC*. Esto es, el valor y la forma asociado al indicador (por ejemplo, un dominio cuyo valor es *officeonline.com*).

Así, no se descarga todo el objeto CTI, ya que esto sería un proceso muy largo y pesado, sino sólo los aspectos relevantes para las fases posteriores del procesado.

Los datos se recuperan mediante ingesta paginada, consumiendo todas las páginas devueltas por *OpenCTI* hasta que no quedan más disponibles. Esto se debe a que *OpenCTI* puede tener del orden de cientos de miles o incluso millones de entidades, por lo que una única llamada a la API no sería suficiente para recuperar todos los datos o, según la configuración de *timeouts*, podría no devolver nada.

Una vez finalizada la extracción, guarda los datos obtenidos en ficheros JSON independientes. Por ejemplo, `threat_actors.json` o `indicators.json`. En este caso, se guardan en el directorio `/opt/data/raw/opencti`. Como referencia, una ejecución de este programa, concretamente el 8 de junio de 2026, carga 55 *threat actors*, 355 *malwares*, 410 *attack patterns*, 515569 *indicators* y 5924 *reports*.

Todo este proceso lo lleva a cabo el script `opencti/ingest.py`.

3.2.1.2. Normalización

Este modulo se encarga de la normalización de los datos brutos extraídos de *OpenCTI*. Transforma los objetos recuperados en documentos textuales homogéneos adecuados para su posterior vectorización. Los datos extraídos de *OpenCTI* se almacenan como JSON bruto, por lo que cada objeto recuperado mantiene su estructura original. Esa representación no es la más adecuada para realizar búsquedas semánticas, ya que los modelos de lenguajes trabajan mejor sobre texto descriptivo que sobre JSON complejos.

Es una capa intermedia entre la ingesta bruta y la generación de *embeddings* para contener, para cada objeto CTI, la información relevante en un formato legible, y más importante, igual para todos los objetos de un mismo tipo para facilitar la búsqueda por parte del LLM al vectorizar el texto. Aunque indicadores y reportes se dejan tal y como están, se incluyen en esta parte la búsqueda de relaciones de cierto tipos de objetos en *OpenCTI* para mejorar la semántica asociada al mismo:

- **Threat Actors:** se incluyen relaciones tipo `uses` para recuperar *Attack Patterns* usados y *Malware* asociado.
- **Malware:** se incluyen también relaciones tipo `uses` para recuperar *Attack Patterns* usados.

- **Attack Patterns:** se incluyen relaciones tipo **uses** para recuperar *Threat Actors* que usan estos patrones y técnicas.

El resultado final del proceso de normalización son documentos estructurados, uniformes para cada tipo de objeto recuperado de *OpenCTI*. Por razones de procesado y eficiencia, se guardarán en formato JSONL (*JSON Lines*), que almacena un objeto JSON por línea, aunque extrayendo el contenido y dejándolo en un formto humanamente legible, quedaría como los ejemplo en el Listado 1.

Esta parte del proceso es llevada a cabo por los scripts **normalizer**, que define los procesos de normalización para cada tipo de objeto y **normalize_all.py**, que lee los ficheros JSON guardados, procesa la normalización de cada uno de estos ficheros y los guarda en su correspondiente fichero JSONL.

```

1 # Threat Actor
2
3 THREAT_ACTOR: <nombre_del_actor>
4 Descripción: <descripción_del_actor>
5
6 TTPs usados:
7 - <TTP_1> (<ID_MITRE_1>)
8 - <TTP_2> (<ID_MITRE_2>)
9 - <TTP_N> (<ID_MITRE_N>)
10
11 Malware asociado:
12 - <malware_1>
13 - <malware_2>
14 - <malware_N>
15
16 Creado: <fecha_de_creación>
17
18
19 # Malware
20
21 MALWARE: <nombre_del_malware>
22 Tipos: <tipo_1>, <tipo_2>, <tipo_N>
23 Descripción: <descripción_del_malware>
24
25 TTPs usados:
26 - <TTP_1> (<ID_MITRE_1>)
27 - <TTP_2> (<ID_MITRE_2>)
28 - <TTP_N> (<ID_MITRE_N>)
29
30 Creado: <fecha_de_creación>
31
32
33 # Attack Pattern / TTP
34
35 TTP: <nombre_de_la_técnica>
36 ID_MITRE: <identificador_MITRE>
37 Descripción: <descripción_de_la_técnica>
38
39 Usado por:
40 - <actor_1>
41 - <actor_2>
42 - <actor_N>
43

```

```

44 Creado: <fecha_de_creación>
45
46
47 # Indicator / IOC
48
49 INDICATOR (IOC)
50 Patrón: <patrón_del_indicador>
51 Tipo: <tipo_de_patrón>
52 Creado: <fecha_de_creación>
53 Válido desde: <fecha_de_inicio_de_validez>
54
55
56 # Report
57
58 REPORT: <nombre_del_report>
59 Descripción: <descripción_del_report>
60 Creado: <fecha_de_creación>

```

Listado 1: Ejemplos de estructuras normalizadas

3.2.1.3. Generación de *embeddings* e inserción en *ChromaDB*

Tras la fase de normalización, los objetos CTI extraídos desde OpenCTI se encuentran representados como documentos textuales semiestructurados. Aunque estos documentos ya son legibles y contienen información relevante sobre actores, malware, técnicas, indicadores e informes, todavía no pueden ser utilizados directamente para realizar búsquedas semánticas. Para ello es necesario transformarlos en representaciones vectoriales, conocidas como *embeddings*, que permiten comparar documentos y consultas en función de su similitud semántica.

El proceso de vectorización consiste en cargar los ficheros `.jsonl` generados durante la normalización, extraer el campo `text` de cada documento y convertirlo en un vector numérico mediante el modelo `sentence-transformers/all-MiniLM-L6-v2`. Este modelo genera una representación densa del significado del texto, de forma que documentos conceptualmente similares quedan próximos dentro del espacio vectorial. En paralelo, también se cargan los ficheros de metadatos asociados a cada colección, comprobando que el número de textos y metadatos coincide antes de generar los embeddings. Esta comprobación es importante para garantizar que cada vector queda asociado al documento y metadatos correctos.

Los embeddings generados se almacenan en ficheros `.npy`, separados por tipo de entidad. Así, se generan ficheros independientes para `threat_actors`, `malware`, `attack_patterns`, `reports` e `indicators`. Esta separación permite mantener una organización coherente del corpus CTI y facilita la carga posterior en colecciones diferenciadas dentro de ChromaDB. Además, guardar los embeddings como ficheros NumPy permite reutilizarlos sin necesidad de recalcularlos cada vez que se quiera reconstruir la base vectorial.

Una vez generados los embeddings, el siguiente paso consiste en cargarlos en ChromaDB. Para ello, el script de inserción se conecta al servidor de ChromaDB mediante `HttpClient`. Para cada tipo de entidad, el script crea o recupera una colección con el mismo nombre de la entidad, por ejemplo `threat_actors`, `malware`, `attack_patterns`, `reports` o `indicators`. Las colecciones se configuran utilizando similitud coseno, una métrica habitual para comparar embeddings en sistemas de recuperación semántica.

Antes de insertar los documentos en ChromaDB, el script valida que los textos normalizados, los metadatos y los embeddings tengan la misma longitud. Esta validación evita desalineaciones entre documentos, vectores y metadatos. Posteriormente, la inserción se realiza por lotes, asociando a cada documento su identificador, su texto, su embedding y sus metadatos. De este modo, cada entrada almacenada en ChromaDB conserva tanto la representación vectorial necesaria para la búsqueda semántica como la información textual y de trazabilidad necesaria para interpretar el resultado recuperado.

El resultado final de este proceso es una base vectorial organizada en colecciones CTI, donde cada documento procedente de OpenCTI puede ser recuperado por similitud semántica. Durante una consulta, el backend RAG genera un embedding de la pregunta o del indicador detectado y consulta las colecciones de ChromaDB para obtener los documentos más cercanos. Estos documentos se incorporan al contexto final como información contextual o inferida, diferenciándola de la inteligencia explícita recuperada directamente desde OpenCTI.

La vectorización y carga en ChromaDB son, por tanto, una parte esencial del sistema RAG. Esta fase permite complementar la información confirmada de OpenCTI con contexto semántico adicional, como actores, malware, técnicas, informes o indicadores relacionados conceptualmente con la consulta. Sin embargo, la similitud vectorial no implica una relación confirmada entre entidades CTI. Por este motivo, los documentos recuperados desde ChromaDB se tratan como contexto inferido y se separan explícitamente de la información considerada confirmada.

Este proceso lo llevan a cabo los scripts *embed_all_st.py*, para la generación de *embeddings*, y *chroma_load.py*

3.2.2. Frontend, backend y motor de RAG

Se puede entender el workflow anterior como una tarea a realizar con la instalación, para el primer poblado de la base de datos vectorial, y periódicamente para mantener el sistema actualizado respecto al conocimiento de OpenCTI. Así, este segundo workflow, que se puede ver etiquetado con números arábigos del 1 al 6 en la Figura 2 y en más detalle en el caso de uso más común para esta aplicación, en el cual el analista haría una pregunta en la interfaz web seleccionando el modelo que quiere que responda, y espera a obtener un análisis del indicador por el que ha preguntado.

3.2.2.1. *OpenWebUI* y servidor *Pipelines*

Estas dos partes se consideran parte del mismo sistema, pese a ser realmente disjuntas y ser distribuibles (*OpenWebUI* al servidor *Pipelines* a través de una IP remota), dado que el servidor *Pipelines* es un plugin del ecosistema *OpenWebUI*, el cual se conecta a dicho servidor y permite mostrar y configurar las *pipelines* configuradas en la propia interfaz.

OpenWebUI es una interfaz web conversacional. Su función es proporcionar al usuario un entorno web desde el que formular preguntas sobre indicadores de compromiso y visualizar las respuestas generadas por los modelos. *OpenWebUI* no contiene lógica de recuperación de contexto y no accede directamente a *OpenCTI* o *ChromaDB*, dado que no puede hacerlo de forma nativa (sí que tiene una funcionalidad de base de conocimiento, pero esta se basa en la subida de archivos *PDF* o texto plano, e incluso

```

/opt
├── chat-feats
│   └── docker-compose.yml
├── modelos_ia
│   ├── all-MiniLM-L6-v2
│   ├── cyberpal
│   │   └── Modelfile
│   ├── lily
│   │   └── Modelfile
│   └── ollama
├── openwebui_data
├── openwebui_pipelines
│   ├── cybeross_pipeline
│   ├── lily_pipeline
│   ├── rag_pipeline
│   ├── cybeross_pipeline.py
│   └── lily_pipeline.py
├── python-scripts
│   ├── opencti
│   │   ├── chroma_load.py
│   │   ├── client.py
│   │   ├── embed_all_st.py
│   │   ├── ingest.py
│   │   ├── normalize_all.py
│   │   ├── normalizer.py
│   │   └── rag_client.py
│   ├── rag
│   │   ├── context
│   │   │   ├── builder.py
│   │   │   └── context_profiles.py
│   │   ├── app.py
│   │   ├── ioc_context_builder.py
│   │   ├── ioc_resolver.py
│   │   └── query_classifier.py
│   ├── main.py
│   └── requirements.txt

```

Figura 3: Estructura de directorios del proyecto

búsqueda en la web, pero no en la consulta de APIs privadas), por lo que delega esa función en el servidor *Pipelines*.

El despliegue de *OpenWebUI* se realiza mediante *Docker*, que facilita su ejecución aislada y su integración con el resto de componenetes del sistema. Dentro de la configuración de este contenedor (que se puede consultar como *docker-compose.yml* en el **Anexo B: Ficheros desarrollados**). Se puede configurar la dirección de *Ollama* con la variable `OLLAMA_HOST`, permitiendo la consulta directa de los modelos listados por *Ollama*, pero

sin RAG. Adicionalmente, accediendo como usuario administrador (que en este caso, como es una plataforma expuesta de forma privada a un equipo concreto, puede ser accesible para todo el mundo configurando la variable `WEBUI_AUTH` como `false`), yendo al *Admin Panel*, después a los ajustes (*Settings*) y a *Pipelines*, se puede configurar la conexión con el servidor *Pipelines*. En este caso, como está dentro de la misma red *Docker* con el nombre `pipelines`, será `http://pipelines:9099`. El flujo de un analista sería simplemente, en la interfaz web, seleccionar el modelo que quiere que conteste, escribir la pregunta y aguardar la respuesta.

El servidor *Pipelines* es el componente que transforma *OpenWebUI* en más que una interfaz genérica, permitiendo desarrollar todo tipo de integraciones. Las *pipelines* se definen como scripts *Python* cargados por el servidor de *Pipelines* de *OpenWebUI*. Cada *pipeline* contiene una clase principal *Pipeline* que implementa el método `pipe`, y una clase *Valves*, que permite definir parámetros configurables desde la interfaz. Estos parámetros pueden visualizarse y modificarse desde *OpenWebUI*, que permite ajustar el comportamiento de la *Pipeline* sin modificar directamente el código.

En *OpenWebUI*, las *pipelines* se utilizan como si fueran modelos seleccionables. Es decir, el usuario no selecciona necesariamente un modelo de lenguaje puro, sino una *pipeline* que aparece en la interfaz como una opción más de conversación. Sin embargo, internamente, la *pipeline* no envía la pregunta directamente al modelo, sino que intercepta la consulta, ejecuta lógica personalizada, solicita contexto al backend RAG y finalmente construye un *prompt* enriquecido para el modelo local.

Se han definido dos *pipelines*: *Lily RAG* y *CyberOSS RAG*. Ambas comparten la misma idea general: reciben la pregunta del usuario desde *OpenWebUI*, extraen el último mensaje de tipo `user`, llaman al backend RAG para recuperar contexto CTI y posteriormente envían la pregunta enriquecida al modelo correspondiente mediante *Ollama*. La diferencia entre ambas *pipelines* radica en el modelo utilizado, el formato concreto de llamada al modelo y el perfil de contexto utilizado.

La clase *Valves* actúa como mecanismo de configuración. En la *pipeline* de *Lily* se definen por defecto el modelo `lily-cyber` y el perfil de contexto `small`, mientras que en el *pipeline* de *CyberOss* se definen el modelo `cybeross` (estos nombres son los identificadores que se les da en *Ollama*) y un perfil de contexto más amplio. Más adelante se comentarán en detalle estos perfiles de contexto.

El método `pipe` constituye el punto de entrada a la *pipeline*, es el método que *OpenWebUI* llama, enviándole todo el cuerpo de la conversación, que contiene los mensajes intercambiados. La *pipeline* extrae los mensajes del parámetro `kwargs` y delega la ejecución principal en el método `run`. Después devuelve el resultado, usando la palabra clave `yield`. Esto se debe a que `yield` convierte la función en un generador, que devuelve valores de forma iterable y permite que quien llama vaya consumiendo la salida. Este es el comportamiento *OpenWebUI* espera.

Ambas *pipelines* incluyen comprobaciones para detectar tareas interans de *OpenWebUI*, que suele lanzar peticiones adicionales al modelo (o *pipelines*) seleccionados para rellenar datos como nombre, etiquetas o resumen del chat o posibles preguntas de *follow-up* para seguir la conversación. Recibir estas tareas además de las preguntas sobre CTI satura a los modelos, se invierte mucho tiempo y recursos y pueden saltar *timeouts* o resultados inesperados, así que cuando la *pipeline* recibe esta clase de peticiones utiliza

salvaguardas, como devolver respuestas mínimas, y evita ejecución de recuperación de contexto inútil.

En cambio, si se recibe una pregunta real sobre CTI del usuario, la pipeline envía una petición al *endpoint* `/query-context` del backend, con la pregunta del usuario y el tamaño del contexto a recuperar. La lógica completa de este backend será explicada en la siguiente sección. Una vez que se devuelve el contexto estructurado, lo incorpora al prompt, el cual varía entre *pipelines*:

- **Lily:** En esta *pipeline*, se pide como mensaje de sistema, que el modelo actúe como un analista CTI, utilice toda la información que se le proporciona, extraiga indicadores y hechos explícitos, separe inteligencia factual (extraída de *OpenCTI*) de información inferida obtenida de similitud vectorial en *ChromaDB*.
- **CyberOSS:** el flujo general de la pipeline es similar, se le dan las mismas instrucciones que a *Lily*, pero incorpora una adaptación específica de formato de *prompt*. Durante las pruebas se observó que *CyberOSS* no respondía correctamente con el formato estándar de chat utilizado por otros modelos (como *Lily*) en *Ollama*. Esto se debe a que utiliza unos tokens específicos (`<|end|><|start|>assistance` o `<|start|>user`) para delimitar las preguntas del usuario y las respuestas del asistente.

Las *pipelines* se definen en el directorio `openwebui_pipelines`, que se sincroniza con el directorio interno del contenedor *pipelines* al reiniciar este último, cargando las *pipelines* y mostrándolas en la interfaz de *OpenWebUI* si son correctas, o moviéndolas a un nuevo directorio `failed` si contienen errores.

3.2.2.2. Backend y motor RAG

El backend constituye el componente encargado de generar el contexto que posteriormente será entregado al modelo de lenguaje. A diferencia de las *pipelines*, este backend no se encarga de comunicarse con los modelos, sino que actúa como un servicio especializado de recuperación y construcción de contexto. Su función es recibir la consulta del usuario, extraer el indicador por el que está preguntando, recuperar información explícita de *OpenCTI*, obtener contexto adicional de *ChromaDB* y devolver un bloque de contexto estructurado.

Se implementa sobre *FastAPI* y expone el *endpoint*. Este endpoint recibe una petición (método `POST`) con al menos dos campos: la propia petición (de ahora en adelante *query*, en el campo homónimo) y en el campo `context_profile` recibe el perfil de contexto que define el nivel de detalle de la información a recuperar. Este diseño permite que los pipelines seleccionen más o menos contexto según el modelo que lo vaya a procesarlo.

El flujo general del backend puede resumirse de la siguiente manera:

1. Pregunta del usuario
2. Extracción del IOC
3. Resolución explícita del IOC en *OpenCTI*
4. Búsqueda semántica en *ChromaDB*
5. Construcción del contexto final

6. Devolución del contexto a la *pipeline*

El backend se mantiene desacoplado de la inferencia del modelo. Esta separación permite usar el mismo backend, mismos endpoints, con diferentes modelos y *pipelines*.

El fichero `app.py` define el servicio *FastAPI* y define el *endpoint* principal del backend. Al recibir una petición en `/query-context`, el backend valida que el cuerpo recibido sea JSON y que contenga una pregunta válida. Si esto no se cumple, se devuelve un error controlado 422. Una vez validada la entrada, se obtiene el perfil de contexto seleccionado. En caso de estar vacío, la opción por defecto es `small`. Este perfil determina cuántos documentos contextuales se recuperan y con qué nivel de detalle se incluirán en el contexto final.

A continuación, se intenta extraer un indicador de compromiso de la pregunta utilizando el módulo `ioc_resolver.py`. Si no se detecta ningún IoC válido, el backend devuelve un error indicando que no se ha encontrado un indicador en la pregunta. Para detectar los indicadores, este módulo utiliza matching de expresiones regulares. Esto se debe a que los indicadores suelen seguir formatos claramente identificables. Por ejemplo, una dirección IPv4, una URL, un dominio o un hash tienen patrones sintácticos bien definidos (4 números del 0/1 al 255 separados por puntos, caracteres hexadecimales un numero determinado de veces...). Esta aproximación resulta más simple, rápida y controlable que usar un modelo de lenguaje o un clasificador para detectar indicadores. El orden de detección es el siguiente:

1. **URL**. es importante que vaya antes que los dominios para evitar que una URL sea etiquetada parcialmente como un dominio.
2. **SHA-256**
3. **SHA-1**
4. **MD5**
5. **IPv4**
6. **Dominio**

Una vez detectado el *IoC*, el módulo `resolve_ioc` consulta *OpenCTI* mediante el cliente `rag_client.py`, una encapsulación del cliente normal (`client.py`) de *OpenCTI* para hacer consultas específicas de RAG. Esta resolución obtiene tres tipos de información: `direct_indicators`, que proceden de búsquedas por valor, `pattern_matches`, que filtran aquellos indicadores cuyo campo `pattern` contiene el *IoC* y `related_reports`, que permiten recuperar informes que mencionan el indicador. Esta información se considera la parte fuerte o explícita del contexto, ya que procede directamente de *OpenCTI*.

A continuación, se construye un texto legible mediante `build_ioc_context`. Este módulo transfiere el objeto recuperado desde *OpenCTI* en un bloque de texto estructurado y legible, convirtiendo datos técnicos en una representación útil para análisis CTI. Combina los indicadores recuperados por búsqueda directa y por coincidencia de patrón. Aplica una deduplicación por clave (valor del indicador, estado, *confidence* y patrón STIX) para evitar mostrar el mismo indicador varias veces en el contexto. Esto se debe a que durante las pruebas se observó que la aparición de elementos duplicados

en el contexto tiende a confundir a modelos pequeños, empobreciendo sus respuestas. Cada indicador se muestra junto a sus campos más relevantes, como valor, tipo, estado (activo/revocado), puntuación, patrón, descripción, validez, u origen.

Además de la consulta a *OpenCTI*, el backend realiza una recuperación semántica contra *ChromaDB*. Para ello, `app.py` utiliza el modelo `all-MiniLM-L6-v2` del framework `sentence-transformers` para convertir la consulta en un *embedding*. Este *embedding* se consulta contra varias colecciones vectoriales:

- `threat_actors`
- `malware`
- `attack_patterns`
- `reports`
- `indicators`

Para cada colección, el backend recupera los documentos más cercanos según similitud vectorial. Estos documentos proceden del proceso de ingesta de *OpenCTI*. Esta recuperación permite aportar contexto adicional al modelo que podría ser semánticamente relevante para la consulta, pero es importante recalcar que esta similitud no equivale a una relación confirmada. Es por esto que esta parte del contexto aparece diferenciada explícitamente de la inteligencia fuerte.

El contexto es filtrado de acuerdo a los ya mencionados perfiles de contexto (definidos en `context_profiles.py`). Estos perfiles controlan cuántos documentos semánticos se incluyen en la respuesta y con qué nivel de detalle se presentan. El perfil `small` está pensado para modelos pequeños con poca capacidad de contexto, como *Lily*. Limita el número de documentos inferidos y utiliza una versión compacta de los mismos. Presenta tipo de objeto recuperado, nombre, descripción y fecha de creación. Aunque es algo pobre, puede aportar algo de utilidad como contexto superficial. El contexto `large` incluye más documentos y mantiene el contenido completo. Está pensado para modelos de mayor capacidad como *CyberOSS*, que pueden procesar más contexto y extraer relaciones más complejas. Inicialmente no se planteó la utilización de perfiles de contexto, pero se introdujo al realizar pruebas con *Lily* y ver que el modelo daba respuestas pobres y apenas analizaba el contexto que se le daba. Si el contexto es demasiado grande, puede incluso no responder por cumplirse el *timeout*.

El módulo `builder.py` se encarga de unir el contexto fuerte extraído de *OpenCTI* con el contexto inferido recuperado de *ChromaDB*. Es el último paso antes de devolver el contexto a la *pipeline*. El módulo recibe tres elementos:

- `ioc_context_text`: contiene la información explícita del *IoC*.
- `context_docs`: contienen documentos recuperados por similitud semántica.
- `context_profile`.

Finalmente, a función `_prepare_contextual_docs` aplica el nivel de detalle solicitado según el perfil de contexto y `build_final_context` construye el bloque de contexto final. En el Listado 2 se pueden

ver dos ejemplos de contexto recuperado y devuelto a la *pipeline*, tanto grande como pequeño para el indicador conocido '*officeonline.com*'.

```
1 CONTEXTO PEQUEÑO
2
3 ### IOC ANALYZED
4 [INFORMACIÓN DE OPENCTI IGUAL PARA AMBOS]
5 ### CONTEXTUAL CTI INFORMATION (INFERRED FROM SIMILARITY)
6 The following CTI-related information is retrieved via semantic
  similarity search.
7 - THREAT_ACTOR: thegentlemen
8   Descripción: 'No description available
9   Creado: 2026-02-18T18:14:10.980Z
10 - THREAT_ACTOR: incransom
11   Descripción: 'No description available
12   Creado: 2026-02-18T18:14:08.911Z
13
14
15 CONTEXTO GRANDE
16
17 ### IOC ANALYZED
18 [INFORMACIÓN DE OPENCTI IGUAL PARA AMBOS]
19 ### CONTEXTUAL CTI INFORMATION (INFERRED FROM SIMILARITY)
20 The following CTI-related information is retrieved via semantic
  similarity search.
21 [01ec22a8-5a4e-4a69-a1ad-297792ee17a8]
22 THREAT_ACTOR: thegentlemen
23 Descripción: 'No description available
24 TTPs usados:
25 - N/A
26 Malware asociado:
27 - N/A
28 Creado: 2026-02-18T18:14:10.980Z
29 [b0e2b7f6-345c-4f33-898e-49d743b1594d]
30 THREAT_ACTOR: incransom
31 Descripción: 'No description available
32 TTPs usados:
33 - N/A
34 Malware asociado:
35 - N/A
36 Creado: 2026-02-18T18:14:08.911Z
37 [88f341f2-aa27-4053-8abf-810771cf215d]
38 THREAT_ACTOR: safepay
39 Descripción: 'No description available
40 TTPs usados:
41 - N/A
42 Malware asociado:
43 - N/A
44 Creado: 2026-03-04T12:28:08.257Z
45 [98f622eb-3335-49d1-9add-05a2b53be38c]
46 THREAT_ACTOR: osiris
47 Descripción: 'No description available
48 TTPs usados:
49 - N/A
50 Malware asociado:
51 - N/A
52 Creado: 2026-03-23T15:29:16.121Z
53 [826caec6-81e5-4c89-aac4-8acdcb4918e7]
```

```

54 THREAT_ACTOR: morpheus
55 Descripción: 'No description available
56 TTPs usados:
57 - N/A
58 Malware asociado:
59 - N/A
60 Creado: 2026-02-27T08:41:03.098Z
61 [3f15a188-fbbb-4ceb-903a-ed550bfcfb8c]
62 MALWARE: PlugX
63 Tipos: N/A
64 Descripción: Malware
65 TTPs usados:
66 - Msiexec ()
67 Malicious File ()
68 Drive-by Compromise ()
69 Spearphishing Link ()
70 Web Protocols ()
71 Malicious Link ()
72 Match Legitimate Name or Location ()
73 Dynamic API Resolution ()
74 Email Accounts ()
75 DLL Search Order Hijacking ()
76 PowerShell ()
77 Creado: 2026-01-16T11:26:15.991Z
78 [6a59d49c-d226-48c2-b617-8efac786a948]
79 MALWARE: ZeroT
80 Tipos: N/A
81 Descripción: Most recently, we have observed the same group targeting
    military and aerospace interests in Russia and Belarus. Since the
    summer of 2016, this group began using a new downloader known as
    ZeroT to install the PlugX remote access Trojan (RAT) and added
    Microsoft Compiled HTML Help (.chm) as one of the initial droppers
    delivered in spear-phishing emails.
82 TTPs usados:
83 - N/A
84 Creado: 2026-01-16T11:26:15.992Z
85 [9e199722-b99a-46ca-9389-6c6e999cc348]
86 MALWARE: LP-Notes
87 Tipos: N/A
88 Descripción: None
89 TTPs usados:
90 - N/A
91 Creado: 2026-01-19T16:25:20.948Z

```

Listado 2: Contextos para 'officeonline.com'

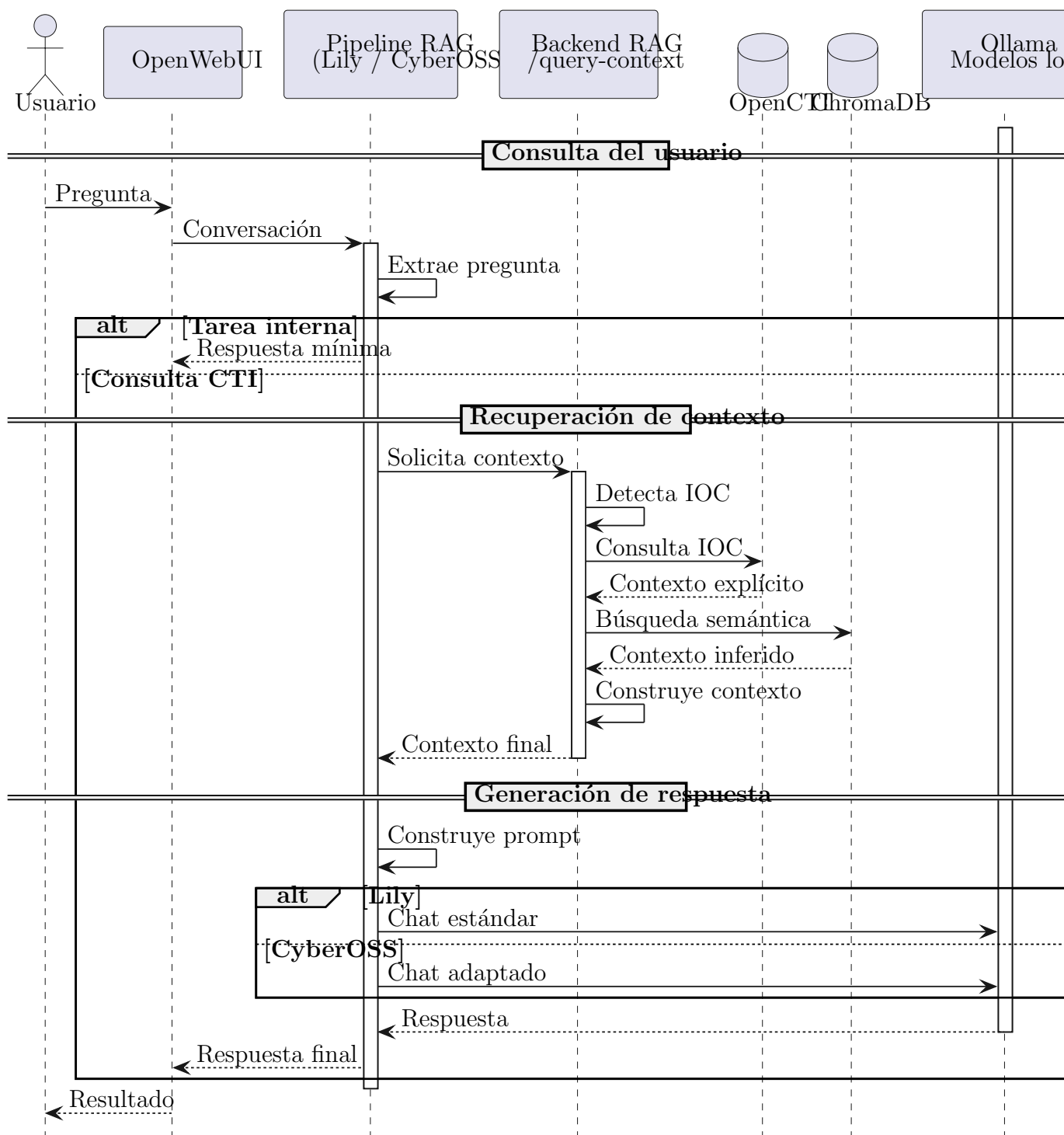


Figura 4: Workflow completo de secuencia

4. Evaluación y discusión de resultados

Pregunta	Lily	CyberOSS
¿Menciona indicadores explícitos?	1	2
¿Muestra bien el estado y confidence?	1	2
Menciona y separa el contexto inferido	0	2
¿Separa relaciones fuertes y débiles?	1	2
¿Alucina?	1	1
¿Es útil para análisis?	1	2
Puntuación	5	11

Tabla 1: Resultados para dominio conocido

Pregunta	Lily	CyberOSS
¿Menciona indicadores explícitos?	1	2
¿Muestra bien el estado y confidence?	1	2
Menciona y separa el contexto inferido	2	2
¿Separa relaciones fuertes y débiles?	1	2
¿Alucina?	1	1
¿Es útil para análisis?	1	2
Puntuación	6	11

Tabla 2: Resultados para dirección IP conocida

Pregunta	Lily	CyberOSS
¿Menciona indicadores explícitos?	2	2
¿Muestra bien el estado y confidence?	2	2
Menciona y separa el contexto inferido	0	2
¿Separa relaciones fuertes y débiles?	1	2
¿Alucina?	2	1
¿Es útil para análisis?	1	2
Puntuación	8	11

Tabla 3: Resultados para hash conocido

Pregunta	Lily	CyberOSS
¿Menciona indicadores explícitos?	0	0
¿Muestra bien el estado y confidence?	0	0
Menciona y separa el contexto inferido	0	0
¿Separa relaciones fuertes y débiles?	0	0
¿Alucina?	0	0
¿Es útil para análisis?	0	0
Puntuación	0	0

Tabla 4: Resultados para pregunta por contexto inferido

Pregunta	Lily	CyberOSS
¿Menciona indicadores explícitos?	1	2
¿Muestra bien el estado y confidence?	0	2
Menciona y separa el contexto inferido	1	2
¿Separa relaciones fuertes y débiles?	0	2
¿Alucina?	0	2
¿Es útil para análisis?	0	2
Puntuación	2	12

Tabla 5: Resultados de relación con actores

5. Conclusiones y trabajo futuro

Pregunta	Lily	CyberOSS
¿Menciona indicadores explícitos?	2	2
¿Muestra bien el estado y confidence?	1	2
Menciona y separa el contexto inferido	2	2
¿Separa relaciones fuertes y débiles?	2	1
¿Alucina?	2	1
¿Es útil para análisis?	2	2
Puntuación	12	10

Tabla 6: Resultados de separación fuerte/débil explícita

Pregunta	Lily	CyberOSS
¿Menciona indicadores explícitos?	2	2
¿Muestra bien el estado y confidence?	2	2
Menciona y separa el contexto inferido	2	2
¿Separa relaciones fuertes y débiles?	2	2
¿Alucina?	2	1
¿Es útil para análisis?	2	2
Puntuación	11	11

Tabla 7: Resultados de pregunta abierta

	Lily	CyberOSS
Promedio	6,29	9,43
Sobre 10	5,24	7,86

Tabla 8: Resultados finales

Anexos

Anexo A: Organización del trabajo.

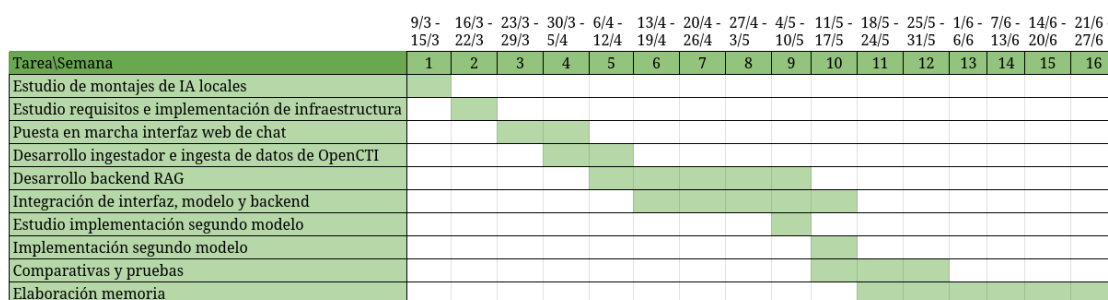


Figura 5: Diagrama de Gantt

Tarea	Horas
Estudio de montajes de IA locales	10
Estudio requisitos e implementación de infraestructura	10
Puesta en marcha interfaz web de chat	20
Desarrollo ingestador e ingesta de datos	25
Desarrollo backend y motor RAG	55
Integración interfaz, modelo y backend	70
Estudio de segundo modelo	5
Implementación segundo modelo	5
Comparativas y pruebas	25
Desarrollo memoria	
Total	225

Tabla 9: Tabla de tareas y tiempos

Anexo B: Ficheros desarrollados.

```

1 # client.py
2 import os
3 from dotenv import load_dotenv
4 from pycti import OpenCTIApiClient
5 import threading
6
7 load_dotenv()
8
9 class ApiClient:
10     # Patrón Singleton para crear UN SOLO cliente API
11     _instance = None
12     _lock = threading.Lock()
13
14     def __new__(cls):
15         if not cls._instance:
16             with cls._lock:
17                 if not cls._instance:
18                     cls._instance = super().__new__(cls)
19         return cls._instance
20
21     def __init__(self):
22         if hasattr(self, "_initialized") and self._initialized:
23             return
24
25         self.url = os.getenv("OPENCTI_URL")
26         self.token = os.getenv("OPENCTI_TOKEN")
27
28         if not self.url or not self.token:
29             raise RuntimeError("Faltan variables OPENCTI_URL o
OPENCTI_TOKEN en .env")
30
31         self.api = OpenCTIApiClient(self.url, self.token)
32
33         print(f"[INFO] ApiClient creado contra {self.url}")
34

```

```
35 self._initialized = True
```

Listado 3: client.py

```
1 # ingest.py
2 import os
3 import json
4 from dotenv import load_dotenv
5 from .client import ApiClient
6
7 load_dotenv()
8
9 DATA_ROOT=os.getenv("DATA_ROOT", "/opt/data")
10 RAW_DIR=f"{DATA_ROOT}/raw/opencti"
11 os.makedirs(RAW_DIR, exist_ok=True)
12
13 class OpenCTIIngestor:
14     def __init__(self):
15         self.api = ApiClient().api
16
17     def _save_json(self, filename, data):
18         path = os.path.join(RAW_DIR, filename)
19
20         with open(path, 'w', encoding="utf-8") as f:
21             json.dump(data, f, indent=2, ensure_ascii=False)
22
23         print(f"[] Guardado en {path} ({len(data)} objetos)")
24
25     def _ingest_entities(self, list_function, label, first=200,
26 custom_attributes=None):
27         """
28         Método general para ingerir entidades desde OpenCTI usando
29         paginación oficial:
30         - withPagination=True
31         - pagination.hasNextPage / endCursor
32         - data["entities"] devuelto por pycti
33         """
34         print(f"\n[+] Iniciando ingesta de {label}...")
35
36         all_entities = []
37         data = {"pagination": {"hasNextPage": True, "endCursor": None}}
38
39         while data["pagination"]["hasNextPage"]:
40             after = data["pagination"]["endCursor"]
41
42             # Llamada oficial según documentación OpenCTI
43             data = list_function(
44                 first=first,
45                 after=after,
46                 customAttributes=custom_attributes,
47                 withPagination=True,
48                 orderBy="created_at",
49                 orderMode="asc",
50             )
51
52             if not data or "entities" not in data:
53                 print(f"[] Error obteniendo datos para {label}")
54                 break
```

```

53         batch = data["entities"]
54         all_entities.extend(batch)
55
56         print(f"        - Descargados {len(batch)} nuevos ({len(
57 all_entities)} total)")
58
59         print(f"[] Ingesta completada: {label} ({len(all_entities)}
60 objetos)")
61         return all_entities
62
63
64     def ingest_threat_actors(self):
65         custom = """
66             id
67             name
68             description
69             created
70         """
71         data = self._ingest_entities(self.api.threat_actor.list, "
72 Threat Actors", 200, custom)
73
74         self._save_json("threat_actors.json", data)
75         return data
76
77     def ingest_malware(self):
78         custom = """
79             id
80             name
81             malware_types
82             description
83             created
84         """
85         data = self._ingest_entities(self.api.malware.list, "Malware",
86 200, custom)
87
88         self._save_json("malware.json", data)
89         return data
90
91     def ingest_attack_patterns(self):
92         custom = """
93             id
94             name
95             x_mitre_id
96             description
97             created
98         """
99         data = self._ingest_entities(self.api.attack_pattern.list, "
100 Attack Patterns", 200, custom)
101
102         self._save_json("attack_patterns.json", data)
103         return data
104
105     def ingest_indicators(self):
106         custom = """
107             id
108             pattern

```

```

106         pattern_type
107         created
108         valid_from
109     """
110     data = self._ingest_entities(self.api.indicator.list, "
Indicators (IOCs)", 200, custom)
111
112     self._save_json("indicators.json", data)
113     return data
114
115
116     def ingest_reports(self):
117         custom = """
118             id
119             name
120             description
121             created
122         """
123         data = self._ingest_entities(self.api.report.list, "Reports",
100, custom)
124
125         self._save_json("reports.json", data)
126         return data
127
128
129     if __name__ == "__main__":
130         ingestor = OpenCTIIngestor()
131
132         actors = ingestor.ingest_threat_actors()
133         malware = ingestor.ingest_malware()
134         ttps = ingestor.ingest_attack_patterns()
135         iocs = ingestor.ingest_indicators()
136         reports = ingestor.ingest_reports()
137
138         print("\n=== RESUMEN FINAL ===")
139         print(f"  Threat Actors: {len(actors)}")
140         print(f"  Malware:         {len(malware)}")
141         print(f"  TTPs:           {len(ttps)}")
142         print(f"  IOCs:           {len(iocs)}")
143         print(f"  Reports:        {len(reports)}")

```

Listado 4: ingest.py

```

1  # normalizer.py
2  from .ingest import OpenCTIIngestor
3  from .client import ApiClient
4  from dotenv import load_dotenv
5
6  load_dotenv()
7
8
9  class OpenCTINormalizer:
10     def __init__(self):
11         self.api = ApiClient().api
12
13     def _get_relationships(self, entity_id, relationship_type=None):
14         relationships = self.api.stix_core_relationship.list(
15             fromId = entity_id,
16             relationship_type = relationship_type,

```

```

17         first=2000
18     )
19     return relationships or []
20
21     def _join_list(self, items):
22         return ", ".join(items) if items else "N/A"
23
24     # Normalizadores individuales de cada tipo de objeto
25     def normalize_threat_actor(self, actor):
26         uses_ttps = self._get_relationships(actor["id"], "uses")
27         uses_malware = [
28             r for r in uses_ttps if r.get("to", {}).get("entity_type")
29             == "Malware"
30         ]
31
32         uses_attack_patterns = [
33             r for r in uses_ttps if r.get("to", {}).get("entity_type")
34             == "Attack-Pattern"
35         ]
36
37         ttps = [
38             f"{r['to']['name']} ({r['to'].get('x_mitre_id','')})"
39             for r in uses_attack_patterns if r.get("to")
40         ]
41
42         malware = [
43             r["to"]["name"] for r in uses_malware if r.get("to")
44         ]
45
46         text = f"""
47         THREAT_ACTOR: {actor.get("name","N/A")}
48         Descripción: {actor.get("description","N/A")}
49
50         TTPs usados:
51         - {chr(10).join(ttps) if ttps else "N/A"}
52
53         Malware asociado:
54         - {chr(10).join(malware) if malware else "N/A"}
55
56         Creado: {actor.get("created","N/A")}
57         """
58
59         return text.strip()
60
61     def normalize_malware(self, malware):
62         relationships = self._get_relationships(malware["id"], "uses")
63
64         ttps = [
65             f"{r['to']['name']} ({r['to'].get('x_mitre_id','')})"
66             for r in relationships if r.get("to",{}).get("entity_type")
67             == "Attack-Pattern"
68         ]
69
70         text = f"""
71         MALWARE: {malware.get("name")}
72         Tipos: {self._join_list(malware.get("malware_types", []))}
73         Descripción: {malware.get("description", "N/A")}

```

```

72 TTPs usados:
73 - {chr(10).join(ttps) if ttps else "N/A"}
74
75 Creado: {malware.get("created")}
76 """
77
78     return text.strip()
79
80
81     def normalize_attack_pattern(self, ttp):
82
83         used_by = self._get_relationships(ttp["id"], "uses")
84         actors = [
85             r["from"]["name"] for r in used_by
86             if r.get("from", {}).get("entity_type") == "Threat-Actor"
87         ]
88
89         text = f"""
90 TTP: {ttp.get("name")}
91 ID_MITRE: {ttp.get("x_mitre_id")}
92 Descripción: {ttp.get("description")}
93
94 Usado por:
95 - {chr(10).join(actors) if actors else "N/A"}
96
97 Creado: {ttp.get("created")}
98 """
99
100     return text.strip()
101
102
103     def normalize_indicator(self, ioc):
104         return f"""
105 INDICATOR (IOC)
106 Patrón: {ioc.get("pattern")}
107 Tipo: {ioc.get("pattern_type")}
108 Creado: {ioc.get("created")}
109 Válido desde: {ioc.get("valid_from")}
110 """.strip()
111
112
113     def normalize_report(self, report):
114         return f"""
115 REPORT: {report.get("name")}
116 Descripción: {report.get("description", "N/A")}
117 Creado: {report.get("created")}
118 """.strip()
119
120
121     def normalize_all_threat_actors(self, actors):
122         return [self.normalize_threat_actor(a) for a in actors]
123
124     def normalize_all_malware(self, malware):
125         return [self.normalize_malware(m) for m in malware]
126
127     def normalize_all_ttps(self, ttps):
128         return [self.normalize_attack_pattern(t) for t in ttps]
129

```

```

130     def normalize_all_reports(self, reports):
131         return [self.normalize_report(r) for r in reports]
132
133     def normalize_all_iocs(self, iocs):
134         return [self.normalize_indicator(i) for i in iocs]
135
136
137     def metadata_threat_actor(self, actor):
138         return {
139             "id": actor.get("id"),
140             "name": actor.get("name"),
141             "type": "Threat Actor"
142         }
143
144     def metadata_malware(self, malware):
145         return {
146             "id": malware.get("id"),
147             "name": malware.get("name"),
148             "type": "Malware"
149         }
150
151     def metadata_ttp(self, ttp):
152         return {
153             "id": ttp.get("id"),
154             "name": ttp.get("name"),
155             "type": "Attack-Pattern"
156         }
157
158     def metadata_report(self, report):
159         return {
160             "id": report.get("id"),
161             "name": report.get("name"),
162             "type": "Report"
163         }
164
165     def metadata_ioc(self, ioc):
166         return {
167             "id": ioc.get("id"),
168             "type": "Indicator"
169         }

```

Listado 5: normalizer.py

```

1  # normalize_all.py
2  import os
3  import json
4  from dotenv import load_dotenv
5
6  from .normalizer import OpenCTINormalizer
7
8  load_dotenv()
9
10 DATA_ROOT = os.getenv("DATA_ROOT", "/opt/data")
11
12 RAW_DIR = f"{DATA_ROOT}/raw/opencti"
13 NORMALIZED_DIR = f"{DATA_ROOT}/normalized"
14
15 os.makedirs(NORMALIZED_DIR, exist_ok=True)
16

```



```

17
18 def load_json(filename):
19     path = os.path.join(RAW_DIR, filename)
20     if not os.path.exists(path):
21         print(f"[!] No existe el fichero: {path}")
22         return []
23     with open(path, "r", encoding="utf-8") as f:
24         return json.load(f)
25
26 def save_normalized_jsonl(filename, texts):
27     path = os.path.join(NORMALIZED_DIR, filename)
28     with open(path, "w", encoding="utf-8") as f:
29         for t in texts:
30             f.write(json.dumps({"text": t}) + "\n")
31         print(f"[ ] Guardado JSONL: {path} ({len(texts)} documentos)")
32
33 def save_metadata(filename, metas):
34     path = os.path.join(NORMALIZED_DIR, filename)
35     with open(path, "w", encoding="utf-8") as f:
36         for m in metas:
37             f.write(json.dumps(m) + "\n")
38         print(f"[ ] Guardado JSONL: {path} ({len(metas)} metadatos)")
39
40 if __name__ == "__main__":
41     normalizer = OpenCTINormalizer()
42
43     print("=== NORMALIZANDO ENTIDADES ===\n")
44
45     actors_raw = load_json("threat_actors.json")
46     actors_text = normalizer.normalize_all_threat_actors(actors_raw)
47     actors_meta = [normalizer.metadata_threat_actor(a) for a in
48 actors_raw]
49
50     save_normalized_jsonl("threat_actors.jsonl", actors_text)
51     save_metadata("threat_actors_meta.jsonl", actors_meta)
52
53     malware_raw = load_json("malware.json")
54     malware_text = normalizer.normalize_all_malware(malware_raw)
55     malware_meta = [normalizer.metadata_malware(m) for m in
56 malware_raw]
57
58     save_normalized_jsonl("malware.jsonl", malware_text)
59     save_metadata("malware_meta.jsonl", malware_meta)
60
61     ttps_raw = load_json("attack_patterns.json")
62     ttps_text = normalizer.normalize_all_ttps(ttps_raw)
63     ttps_meta = [normalizer.metadata_ttp(t) for t in ttps_raw]
64
65     save_normalized_jsonl("attack_patterns.jsonl", ttps_text)
66     save_metadata("attack_patterns_meta.jsonl", ttps_meta)
67
68
69     iocs_raw = load_json("indicators.json")
70     iocs_text = normalizer.normalize_all_iocs(iocs_raw)
71     iocs_meta = [normalizer.metadata_ioc(i) for i in iocs_raw]
72

```

```

73     save_normalized_jsonl("indicators.jsonl", iocs_text)
74     save_metadata("indicators_meta.jsonl", iocs_meta)
75
76
77     reports_raw = load_json("reports.json")
78     reports_text = normalizer.normalize_all_reports(reports_raw)
79     reports_meta = [normalizer.metadata_report(r) for r in reports_raw
80 ]
81
82     save_normalized_jsonl("reports.jsonl", reports_text)
83     save_metadata("reports_meta.jsonl", reports_meta)
84
85     print("=== NORMALIZACIÓN COMPLETADA ===\n")

```

Listado 6: normalize_all.py

```

1  # embed_all_st.py
2  import os
3  import json
4  import numpy as np
5  from sentence_transformers import SentenceTransformer
6
7  DATA_ROOT = os.getenv("DATA_ROOT", "/opt/data")
8
9  NORMALIZED_DIR = f"{DATA_ROOT}/normalized"
10 EMB_DIR = f"{DATA_ROOT}/embeddings"
11
12 os.makedirs(EMB_DIR, exist_ok=True)
13
14 def load_texts_jsonl(filename):
15     path = os.path.join(NORMALIZED_DIR, filename)
16     texts = []
17     with open(path, "r", encoding="utf-8") as f:
18         for line in f:
19             texts.append(json.loads(line)["text"])
20     return texts
21
22
23 def load_metadata(filename):
24     path = os.path.join(NORMALIZED_DIR, filename)
25     metas = []
26     with open(path, "r", encoding="utf-8") as f:
27         for line in f:
28             metas.append(json.loads(line))
29     return metas
30
31 def save_embeddings(filename, vecs):
32     path = os.path.join(EMB_DIR, filename)
33     np.save(path, vecs)
34     print(f"[] Guardado embeddings: {path} shape={vecs.shape}")
35
36 def check_alignment(texts, metas, entity_type):
37     if len(texts) != len(metas):
38         raise ValueError(
39             f"[ERROR] Desalineación en {entity_type}: {len(texts)}
40             textos vs {len(metas)} metadatos"
41         )
42     print(f"[] Alineación OK para {entity_type}: {len(texts)} items")

```

```

43
44 if __name__ == "__main__":
45     print("[+] Cargando modelo 'sentence-transformers/all-MiniLM-L6-v2
46     ...")
47     model = SentenceTransformer("sentence-transformers/all-MiniLM-L6-
48     v2")
49
50     # Threat Actors
51     actors_text = load_texts_jsonl("threat_actors.jsonl")
52     actors_meta = load_metadata("threat_actors_meta.jsonl")
53     check_alignment(actors_text, actors_meta, "Threat Actors")
54
55     emb = model.encode(actors_text, batch_size=64, show_progress_bar=
56     True)
57     save_embeddings("threat_actors.npy", emb)
58
59     # Malware
60     malware_text = load_texts_jsonl("malware.jsonl")
61     malware_meta = load_metadata("malware_meta.jsonl")
62     check_alignment(malware_text, malware_meta, "Malware")
63
64     emb = model.encode(malware_text, batch_size=64, show_progress_bar=
65     True)
66     save_embeddings("malware.npy", emb)
67
68     # Attack Patterns
69     ttps_text = load_texts_jsonl("attack_patterns.jsonl")
70     ttps_meta = load_metadata("attack_patterns_meta.jsonl")
71     check_alignment(ttps_text, ttps_meta, "TTPs")
72
73     emb = model.encode(ttps_text, batch_size=64, show_progress_bar=
74     True)
75     save_embeddings("attack_patterns.npy", emb)
76
77     # Reports
78     reports_text = load_texts_jsonl("reports.jsonl")
79     reports_meta = load_metadata("reports_meta.jsonl")
80     check_alignment(reports_text, reports_meta, "Reports")
81
82     emb = model.encode(reports_text, batch_size=64, show_progress_bar=
83     True)
84     save_embeddings("reports.npy", emb)
85
86     # Indicators
87     iocs_text = load_texts_jsonl("indicators.jsonl")
88     iocs_meta = load_metadata("indicators_meta.jsonl")
89     check_alignment(iocs_text, iocs_meta, "IOCs")
90
91     emb = model.encode(iocs_text, batch_size=256, show_progress_bar=
92     True)
93     save_embeddings("indicators.npy", emb)

```

Listado 7: embed_all_st.py

```

1 # chroma_load.py

```

```

2 import os
3 import json
4 import numpy as np
5 from chromadb import HttpClient
6 from chromadb.config import Settings
7
8 DATA_ROOT = os.getenv("DATA_ROOT", "/opt/data")
9
10 NORMALIZED_DIR = f"{DATA_ROOT}/normalized"
11 EMB_DIR = f"{DATA_ROOT}/embeddings"
12
13 CHROMA_HOST = os.getenv("CHROMADB_HOST", "localhost")
14 CHROMA_PORT = int(os.getenv("CHROMADB_PORT", "8000"))
15
16 chroma = HttpClient (
17     host=CHROMA_HOST,
18     port=CHROMA_PORT
19 )
20
21 def load_texts_jsonl(filename):
22     path = os.path.join(NORMALIZED_DIR, filename)
23     texts = []
24     with open(path, "r", encoding="utf-8") as f:
25         for line in f:
26             texts.append(json.loads(line)["text"])
27     return texts
28
29 def load_metadata(filename):
30     path = os.path.join(NORMALIZED_DIR, filename)
31     meta = []
32     with open(path, "r", encoding="utf-8") as f:
33         for line in f:
34             meta.append(json.loads(line))
35     return meta
36
37 def load_embeddings(filename):
38     path = os.path.join(EMB_DIR, filename)
39     return np.load(path)
40
41
42 def create_or_get_collection(name):
43     try:
44         return chroma.get_collection(name=name)
45     except:
46         return chroma.create_collection(
47             name=name,
48             metadata={"hnsw:space": "cosine"}
49         )
50
51 def insert_into_chroma(collection, embeddings, documents, metadata,
52     batch=1000):
53     print(f"[+] Insertando documento en lotes de {batch}...")
54     total = len(documents)
55
56     for i in range(0, total, batch):
57         end = i + batch
58         batch_embeddings = embeddings[i:end].tolist()
59         batch_docs = documents[i:end]

```

```

59     batch_meta = metadata[i:end]
60     batch_ids = [m["id"] for m in batch_meta]
61
62     collection.add(
63         ids=batch_ids,
64         embeddings=batch_embeddings,
65         documents=batch_docs,
66         metadatas=batch_meta
67     )
68
69     print(f"        {end}/{total} insertados")
70
71
72 if __name__ == "__main__":
73     print("=== CARGA A CHROMADB ===")
74
75     tipos = ["threat_actors", "malware", "attack_patterns", "reports",
76             "indicators"]
77
78     for t in tipos:
79         print(f"\n=== {t.upper()} ===")
80
81         collection = create_or_get_collection(t)
82
83         texts = load_texts_jsonl(f"{t}.jsonl")
84         meta = load_metadata(f"{t}_meta.jsonl")
85         emb = load_embeddings(f"{t}.npz")
86
87         if len(texts) != len(meta) or len(texts) != emb.shape[0]:
88             raise ValueError(
89                 f"[ERROR] Desalineación en {t}: {len(texts)}
90                 textos, {len(meta)} metadatos, {emb.shape[0]} embeddings"
91             )
92
93         insert_into_chroma(
94             collection,
95             embeddings=emb,
96             documents=texts,
97             metadata=meta,
98             batch = 1000 if t != "indicators" else 5000
99         )
100
101     print(f"[] {t} cargados.\n")

```

Listado 8: chroma_load.py

```

1 services:
2   chromadb:
3     image: chromadb/chroma:latest
4     container_name: nodo_ia_chroma
5     networks:
6       - red_ciberinteligencia_privada
7     ports:
8       - "8000:8000"
9     volumes:
10      - /opt/chroma_data:/data
11
12 openwebui:
13   image: ghcr.io/open-webui/open-webui:main

```

```

14     container_name: nodo_ia_openwebui
15     restart: always
16     networks:
17         - red_ciberinteligencia_privada
18     ports:
19         - "8080:8080"
20     environment:
21         OLLAMA_HOST: "http://[protegido]:11434"
22         WEBUI_AUTH: "false"
23     volumes:
24         - /opt/openwebui_data:/app/backend/data
25     depends_on:
26         - chromadb
27         - pipelines
28
29     pipelines:
30         image: ghcr.io/open-webui/pipelines:main
31         container_name: nodo_ia_pipelines
32         restart: always
33         networks:
34             - red_ciberinteligencia_privada
35         ports:
36             - "9099:9099"
37         environment:
38             PYTHONBUFFERED: 1
39         volumes:
40             - /opt/openwebui_pipelines:/app/pipelines
41         extra_hosts:
42             - "host.docker.internal:host-gateway"
43
44     networks:
45         red_ciberinteligencia_privada:
46             driver: bridge
47
48     volumes:
49         chroma_data:
50         chat_mongo_data:
51         openwebui_data:

```

Listado 9: docker-compose.yml

```

1 # lily_pipeline.py
2 from pydantic import BaseModel
3 import requests
4
5 OLLAMA_HOST = "http://[protegido]:11434"
6 RAG_API_URL = "http://172.17.0.1:9000/query-context"
7
8
9 class Valves(BaseModel):
10     model: str = "lily-cyber"
11     context_profile: str = "small"
12
13 class Pipeline:
14     id = "lily-rag"
15     name = "Lily RAG"
16
17     valves = Valves()
18

```

```

19     def pipe(self, body, **kwargs):
20         messages = body.get("messages", [])
21
22         kwargs.pop("messages", None)
23         kwargs.pop("model", None)
24
25         result = self.run(messages=messages, **kwargs)
26
27         yield result
28
29     def run(self, messages=None, model=None, **kwargs):
30         valves = kwargs.get("valves") or self.valves
31
32         selected_model, context_profile = self._get_config(kwargs)
33
34         print(f"[Lily pipeline] Selected model: {selected_model}",
35 flush=True)
36         print(f"[Lily pipeline] Context profile: {context_profile}",
37 flush=True)
38
39         user_query = self._extract_user_query(messages)
40         if not user_query:
41             return {
42                 "role": "assistant",
43                 "content": "No input"
44             }
45
46         internal_response = self._handle_openwebui_task(user_query)
47         if internal_response is not None:
48             print("[Lily pipeline] OpenWebUI internal task detected,
49 bypassing RAG", flush=True)
50             return internal_response
51
52         rag_context = self._get_rag_context(
53             query=user_query,
54             context_profile=context_profile
55         )
56
57         final_messages = self._build_lily_messages(
58             messages=messages,
59             rag_context=rag_context,
60             user_query=user_query
61         )
62
63         data = self._call_lily(
64             model=selected_model,
65             messages=final_messages
66         )
67
68         return data["message"]["content"]
69
70     def _get_config(self, kwargs):
71         valves = kwargs.get("valves") or self.valves
72
73         if isinstance(valves, dict):
74             selected_model = valves.get("model") or self.valves.model
75             context_profile = valves.get("context_profile") or self.
76 valves.context_profile

```

```

73         else:
74             selected_model = valves.model
75             context_profile = valves.context_profile
76
77         return selected_model, context_profile
78
79     def _extract_user_query(self, messages):
80         user_query = ""
81
82         for msg in reversed(messages):
83             if msg.get("role") == "user":
84                 user_query = msg.get("content", "")
85                 break
86
87         return user_query
88
89
90     def _get_rag_context(self, query, context_profile):
91         resp = requests.post(
92             RAG_API_URL,
93             json={
94                 "query": query,
95                 "context_profile": context_profile
96             },
97             timeout=180
98         )
99
100         resp.raise_for_status()
101
102         return resp.json()["context"]
103
104     def _build_lily_messages(self, messages, rag_context, user_query):
105         return [
106             {
107                 "role": "system",
108                 "content": self._build_lily_system_prompt(
109                     rag_context=rag_context,
110                     user_query=user_query
111                 )
112             }
113         ] + messages
114
115     def _build_lily_system_prompt(self, rag_context, user_query):
116         return f"""
117 You are a cyber threat intelligence analyst.
118
119 Use ALL the information provided in the context.
120
121 Instructions:
122     - First, extract ALL explicit indicators and facts
123     - Then. describe all CTI entities mentioned, including threat
124       actors, reports, malware, campaigns, techniques, infrastructure or
125       any other CTI objects
126     - Then, explain what relationships are clearly prsent and what
127       remains uncertain.
128
129 Rules:
130     - Do NOT invent information

```



```

128     - Do NOT ignore any part of the context
129     - If something is unclear, say it clearly.
130     - Clearly distinguish factual OpenCTI intelligence from inferred/
contextual information.
131     - Write a clear, natural and structured explanation.
132
133     ### CONTEXT
134     {rag_context}
135
136     ### QUESTION
137     {user_query}
138     """
139
140     def _call_lily(self, model, messages):
141         response = requests.post(
142             f"{OLLAMA_HOST}/api/chat",
143             json={
144                 "model": model,
145                 "messages": messages,
146                 "stream": False
147             },
148             timeout=300
149         )
150
151         response.raise_for_status()
152         return response.json()
153
154     def _handle_openwebui_task(self, user_query: str):
155         q = (user_query or "").lower()
156
157         if (
158             "follow_ups" in q
159             or "follow-up" in q
160             or "follow up" in q
161             or "relevant follow-up" in q
162         ):
163             return '{"follow_ups": []}'
164
165         if (
166             "tags" in q
167             or "generate tags" in q
168             or "suggest tags" in q
169             or "chat tags" in q
170         ):
171             return '{"tags": []}'
172
173         if (
174             "generate a concise" in q
175             and "title" in q
176         ):
177             return "Análisis indicador CTI"
178
179         if (
180             "<chat_history>" in q
181             or "chat_history" in q
182             or "summarize the conversation" in q
183             or "conversation summary" in q
184         ):

```

```

185         return ""
186
187     return None

```

Listado 10: lily_pipeline.py

```

1  # cybeross_pipeline.py
2  from pydantic import BaseModel
3  import requests
4
5
6  OLLAMA_HOST = "http://[protegido]:11434"
7  RAG_API_URL = "http://172.17.0.1:9000/query-context"
8
9  class Valves(BaseModel):
10     model: str = "cybeross"
11     context_profile: str = "large"
12
13  class Pipeline:
14     id = "cybeross-rag"
15     name = "CyberOSS RAG"
16
17     valves = Valves()
18
19     def pipe(self, body, **kwargs):
20         messages = body.get("messages", [])
21
22         kwargs.pop("messages", None)
23         kwargs.pop("model", None)
24
25         result = self.run(messages=messages, **kwargs)
26
27         yield result
28
29     def run(self, messages=None, model=None, **kwargs):
30         messages = messages or []
31
32         selected_model, context_profile = self._get_config(kwargs)
33
34         print(f"[CyberOSS pipeline] Selected model: {selected_model}",
35               flush = True)
36         print(f"[CyberOSS pipeline] Context profile: {context_profile}",
37               flush = True)
38
39         user_query = self._extract_last_user_message(messages)
40
41         if not user_query:
42             return "No input"
43
44         internal_response = self._handle_openwebui_task(user_query)
45         if internal_response is not None:
46             print("[Lily pipeline] OpenWebUI internal task detected,
47                   bypassing RAG", flush=True)
48             return internal_response
49
50         rag_context = self._get_rag_context(
51             query=user_query,
52             context_profile=context_profile

```

```

51     )
52
53     cybeross_prompt = self._build_cybeross_prompt(
54         rag_context=rag_context,
55         user_query=user_query
56     )
57
58     answer = self._call_cybeross(
59         model=selected_model,
60         prompt=cybeross_prompt
61     )
62
63     return answer
64
65     def _get_config(self, kwargs):
66         valves = kwargs.get("values") or self.valves
67
68         if isinstance(valves, dict):
69             selected_model = valves.get("model") or self.valves.model
70             context_profile = valves.get("context_profile") or self.
71 valves.context_profile
72         else:
73             selected_model = valves.model
74             context_profile = valves.context_profile
75
76         return selected_model, context_profile
77
78     def _extract_last_user_message(self, messages):
79         for msg in reversed(messages):
80             if msg.get("role") == "user":
81                 return msg.get("content", "")
82         return ""
83
84     def _get_rag_context(self, query, context_profile):
85         response = requests.post(
86             RAG_API_URL,
87             json={
88                 "query": query,
89                 "context_profile": context_profile
90             },
91             timeout=180
92         )
93
94         response.raise_for_status()
95
96         return response.json()["context"]
97
98     def _build_cybeross_prompt(self, rag_context, user_query):
99         analysis_prompt = f"""
100 You are a cyber threat intelligence analyst.
101 Use ALL the information provided in the context.
102
103 Instructions:
104 - First, extract ALL explicit indicators and facts
105 - Then, describe all CTI entities mentioned, including threat actors,
106     reports, malware, campaigns, techniques, infrastructure or any
107     other CTI objects

```

```

106 - Then, explain what relationships are clearly present and what remains
    uncertain.
107
108 Rules:
109 - Do NOT invent information
110 - Do NOT ignore any part of the context
111 - If something is unclear, say it clearly.
112 - Clearly distinguish factual OpenCTI intelligence from inferred/
    contextual information.
113 - Write a clear, natural and structured explanation.
114
115 ### CONTEXT
116 {rag_context}
117
118 ### QUESTION
119 {user_query}
120 """.strip()
121
122     return self._wrap_for_cybeross(analysis_prompt)
123
124     def _wrap_for_cybeross(self, text):
125         return f"""<|start|>user
126 {text}
127 <|end|>
128 <|start|>assistant
129 """
130
131     def _call_cybeross(self, model, prompt):
132         response = requests.post(
133             f"{OLLAMA_HOST}/api/chat",
134             json={
135                 "model": model,
136                 "messages": [
137                     {
138                         "role": "user",
139                         "content": prompt
140                     }
141                 ],
142                 "stream": False,
143                 "options": {
144                     "temperature": 0.2,
145                     "stop": [
146                         "<|end|>",
147                         "<|start|>user"
148                     ]
149                 }
150             },
151             timeout=300
152         )
153
154         response.raise_for_status()
155
156         data = response.json()
157
158         return data["message"]["content"].strip()
159
160     def _handle_openwebui_task(self, user_query: str):
161         q = (user_query or "").lower()

```

```

162
163     if (
164         "follow_ups" in q
165         or "follow-up" in q
166         or "follow up" in q
167         or "relevant follow-up" in q
168     ):
169         return '{"follow_ups": []}'
170
171     if (
172         "tags" in q
173         or "generate tags" in q
174         or "suggest tags" in q
175         or "chat tags" in q
176     ):
177         return '{"tags": []}'
178
179     if (
180         "generate a concise" in q
181         and "title" in q
182     ):
183         return "Análisis indicador CTI"
184
185     if (
186         "<chat_history>" in q
187         or "chat_history" in q
188         or "summarize the conversation" in q
189         or "conversation summary" in q
190     ):
191         return ""
192
193     return None

```

Listado 11: cybeross_pipeline.py

```

1 # app.py
2 from fastapi import FastAPI, Request, HTTPException
3 import json
4 from typing import Dict, Optional, Any, List
5 import os
6 import logging
7
8 from sentence_transformers import SentenceTransformer
9 from chromadb import HttpClient
10
11 from rag.ioc_resolver import extract_ioc_value, resolve_ioc
12 from rag.ioc_context_builder import build_ioc_context
13 from opencti.rag_client import OpenCTIRAGClient
14 from rag.context_builder import build_final_context
15
16 logger = logging.getLogger(__name__)
17 opencti_rag_client = OpenCTIRAGClient()
18
19 CHROMADB_HOST = os.getenv("CHROMADB_HOST", "localhost")
20 CHROMADB_PORT = int(os.getenv("CHROMADB_PORT", "8000"))
21
22 TOP_K = int(os.getenv("TOP_K", "5"))
23
24 COLLECTIONS = [

```

```

25     "threat_actors",
26     "malware",
27     "attack_patterns",
28     "reports",
29     "indicators"
30 ]
31
32 app = FastAPI(
33     title="CTI RAG Server",
34     description="RAG backend sobre OpenCTI usando ChromaDB y
35     embeddings MiniLM",
36     version="1.0.0"
37 )
38
39 embedder = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2
40 ")
41
42 chroma = HttpClient(
43     host=CHROMADB_HOST,
44     port=CHROMADB_PORT
45 )
46
47 def embed_query(text:str) -> List[float]:
48     return embedder.encode([text])[0].tolist()
49
50 def retrieve_context(query: str) -> List[str]:
51     vector = embed_query(query)
52     context_docs = []
53
54     for collection_name in COLLECTIONS:
55         try:
56             collection = chroma.get_collection(collection_name)
57             res = collection.query(
58                 query_embeddings=[vector],
59                 n_results=TOP_K
60             )
61             for doc, meta in zip(res["documents"][0], res["metadatas"]
62 ] [0]):
63                 doc_id = meta.get("id", "unknown")
64                 context_docs.append(f"[{doc_id}]\n{doc}")
65         except Exception:
66             continue
67     return context_docs
68
69 @app.post("/query-context", tags=["RAG"])
70 async def query_rag(request: Request):
71     raw_body = await request.body()
72
73     try:
74         payload = json.loads(raw_body)
75         print(json.dumps(payload, indent=2))
76     except Exception:
77         raise HTTPException(
78             status_code=400,
79             detail="El body recibido no es JSON válido"

```

```

80         )
81
82     question = payload.get("query")
83
84
85     if not isinstance(question, str) or not question.strip():
86         raise HTTPException(
87             status_code=422,
88             detail="No se recibió una pregunta válida"
89         )
90
91     question = question.strip()
92
93     context_profile = payload.get("context_profile", "small")
94
95     # Obtención de contexto por ChromaDB
96
97     ioc_context = None
98     ioc_context_text = None
99
100     ioc_value = extract_ioc_value(question)
101
102     if not ioc_value:
103         raise HTTPException(
104             status_code=422,
105             detail="No se encontró ningún indicador válido en la
pregunta"
106         )
107
108     print(f"[RAG] Detected IOC value: {ioc_value}")
109
110
111     # resolve_ioc devuelve en un objeto el ioc y lo que encuentre por
coincidencia directa de valor, patrón o mención
112     ioc_context = resolve_ioc(ioc_value, opencti_rag_client)
113
114     print("[RAG] IOC RESOLUTION RESULT:")
115     print(ioc_context)
116
117     #build_ioc_context recibe el objeto de contexto creado antes y
devuelve como string formateado el contexto objetivo
118     ioc_context_text = build_ioc_context(ioc_context)
119
120     context_docs = retrieve_context(f"{ioc_value} threat actor malware
infrastructure")
121
122     if not context_docs and not ioc_context_text:
123         raise HTTPException(
124             status_code=404,
125             detail="No se encontró contexto relevante en OpenCTI"
126         )
127
128
129     final_context = build_final_context(
130         ioc_context_text=ioc_context_text,
131         context_docs=context_docs,
132         context_profile=context_profile
133     )

```

```

134
135     print("[DEBUG] Final context returned")
136     print(final_context)
137     print("[DEBUG] End context")
138
139     return {
140         "context": final_context
141     }

```

Listado 12: app.py

```

1  # ioc_resolver.py
2  from typing import Dict, List
3  import re
4
5  def extract_ioc_value(question: str) -> str:
6      url_match = re.search(r"https?:/[^\s\''<>]+", question)
7      if url_match:
8          print(f"URL matchheada: {url_match.group(0)}")
9          return url_match.group(0)
10
11      sha256_match = re.search(r"\b[a-fA-F0-9]{64}\b", question)
12      if sha256_match:
13          print(f"SHA256 matchheada: {sha256_match.group(0)}")
14          return sha256_match.group(0)
15
16      sha1_match = re.search(r"\b[a-fA-F0-9]{40}\b", question)
17      if sha1_match:
18          print(f"SHA1 matchheada: {sha1_match.group(0)}")
19          return sha1_match.group(0)
20
21      md5_match = re.search(r"\b[a-fA-F0-9]{32}\b", question)
22      if md5_match:
23          print(f"MD5 matchheada: {md5_match.group(0)}")
24          return md5_match.group(0)
25
26      ip_match = re.search(
27          r"\b(?:?:25[0-5]|2[0-4]\d|[01]?\d\d?)\.(?:25[0-5]|2[0-4]\d|[01]?\d\d?)\b",
28          question,
29      )
30
31      if ip_match:
32          print(f"IPv4 matchheada: {ip_match.group(0)}")
33          return ip_match.group(0)
34
35      domain_match = re.search(r"\b[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}\b",
36          question)
37      if domain_match:
38          print(f"Dominio matchheada: {domain_match.group(0)}")
39          return domain_match.group(0)
40
41      return ""
42
43  def resolve_ioc(ioc_value: str, opencti_client) -> Dict:
44      result = {
45          "ioc_value": ioc_value,
46          "direct_indicators": [],
47          "pattern_matches": [],

```



```

48     "related_reports": [],
49 }
50
51 direct = opencti_client.find_indicators_by_value(ioc_value)
52 result["direct_indicators"] = direct
53
54 pattern_matches = opencti_client.find_indicators_by_pattern(
55 ioc_value)
56 result["pattern_matches"] = pattern_matches
57
58 reports = opencti_client.find_reports_mentioning(ioc_value)
59 result["related_reports"] = reports
60
61 return result

```

Listado 13: ioc_resolver.py

```

1 # rag_client.py
2 from typing import List, Dict
3 from opencti.client import ApiClient
4
5 """
6 Cliente que utiliza el cliente de OpenCTI para hacer las búsquedas
7 necesarias para el RAG
8 """
9
10 class OpenCTIRAGClient:
11     def __init__(self):
12         self.client = ApiClient().api
13
14     def find_indicators_by_value(self, value: str) -> List[Dict]:
15         return self.client.indicator.list(
16             search=value,
17             first=20,
18         )
19
20     def find_indicators_by_pattern(self, value: str) -> List[Dict]:
21         indicators = self.find_indicators_by_value(value)
22
23         return [
24             i for i in indicators
25             if i.get("pattern") and value in i["pattern"]
26         ]
27
28     def get_indicator_relationships(self, indicator_id: str) -> List[
29 Dict]:
30         return self.client.stix_core_relationship.list(
31             fromId=indicator_id,
32             first=50,
33         )
34
35     def find_reports_mentioning(self, value: str) -> List[Dict]:
36         return self.client.report.list(
37             search=value,
38             first=10,
39         )

```

40

)

Listado 14: rag_client.py

```

1 # ioc_context_builder.py
2 from typing import Dict, List
3
4 def _summarize_indicators(indicators: List[Dict]) -> List:
5     summaries = []
6
7     for ind in indicators:
8         value = ind.get("name", "unknown")
9         pattern = ind.get("pattern", "")
10        confidence = ind.get("confidence", "N/A")
11        revoked = ind.get("revoked", False)
12
13        status = "Revoked" if revoked else "Active"
14
15        summaries.append(
16            f"- {value} ({status}, confidence: {confidence})"
17        )
18    return summaries
19
20 def _summarize_reports(reports: List[Dict]) -> List:
21     summaries = []
22
23     for rpt in reports:
24         name = rpt.get("name", "Unnamed report")
25         published = rpt.get("published", "unknown")
26         confidence = rpt.get("confidence", "N/A")
27
28         summaries.append(
29             f"- {name} (published: {published}, confidence: {confidence})"
30         )
31
32    return summaries
33
34 def _safe(value, default="N/A"):
35     if value is None:
36         return default
37     if value == "":
38         return default
39     return value
40
41 def _indicator_value(indicator: Dict) -> str:
42     return (
43         indicator.get("name")
44         or indicator.get("value")
45         or indicator.get("observable value")
46         or "unknown"
47     )
48
49 def _indicator_status(indicator: Dict) -> str:
50     return "Revoked" if indicator.get("revoked") else "Active"
51
52 def _indicator_type(indicator: Dict) -> str:
53     return (
54         indicator.get("x_opencti_main_observable_type")

```

```

55         or indicator.get("entity_type")
56         or "Unknown"
57     )
58
59 def _source_org(indicator: Dict) -> str:
60     created_by = indicator.get("createdBy") or {}
61     return created_by.get("name") or "N/A"
62
63 def _external_references(indicator: Dict) -> str:
64     refs = indicator.get("externalReferences") or []
65     output = []
66
67     for ref in refs:
68         description = ref.get("description")
69         source_name = ref.get("source_name")
70         url = ref.get("url")
71
72         parts = []
73
74         if source_name:
75             parts.append(source_name)
76
77         if description:
78             parts.append(description)
79
80         if url:
81             parts.append(url)
82
83         if parts:
84             output.append(" | ".join(parts))
85
86     return output
87
88 def _deduplicate_indicators(indicators: list[Dict]) -> list[Dict]:
89     seen = set()
90     result = []
91
92     for indicator in indicators:
93         value = _indicator_value(indicator)
94         status = _indicator_status(indicator)
95         confidence = indicator.get("confidence")
96         pattern = indicator.get("pattern")
97
98         key = (value, status, confidence, pattern)
99
100        if key not in seen:
101            seen.add(key)
102            result.append(indicator)
103
104    return result
105
106 def _render_indicator(indicator: Dict) -> str:
107     value = _indicator_value(indicator)
108     status = _indicator_status(indicator)
109     observable_type = _indicator_type(indicator)
110
111     lines = [
112         f"- Indicator: {value}",

```

```

113         f" Type: {_safe(observable_type)}"
114         f" Status: {status}"
115         f" Confidence: {_safe(indicator.get('confidence'))}"
116     ]
117
118     score = indicator.get("x_opencti_score")
119     if score is not None:
120         lines.append(f" Score: {score}")
121
122     detection = indicator.get("x_opencti_detection")
123     if detection is not None:
124         lines.append(f" Detection: {detection}")
125
126     pattern = indicator.get("pattern")
127     if pattern:
128         lines.append(f" Pattern: {pattern}")
129
130     description = indicator.get("description")
131     if description:
132         lines.append(f" Description: {description}")
133
134     valid_from = indicator.get("valid_from")
135     if valid_from:
136         lines.append(f" Valid from: {valid_from}")
137
138     valid_until = indicator.get("valid_until")
139     if valid_until:
140         lines.append(f" Valid until: {valid_until}")
141
142     source_org = _source_org(indicator)
143     if source_org != "N/A":
144         lines.append(f" Source organization: {source_org}")
145
146     markings = indicator.get("objectMarking") or []
147     if markings:
148         marking_values = [
149             m.get("definition")
150             for m in markings
151             if m.get("definition")
152         ]
153         if marking_values:
154             lines.append(f" Marking: {' '.join(marking_values)}")
155
156     refs = _external_references(indicator)
157     if refs:
158         lines.append(" External references:")
159         for ref in refs[:3]:
160             lines.append(f" - {ref}")
161
162     return "\n".join(lines)
163
164 def build_ioc_context(ioc_context: Dict | None) -> str | None:
165     if not ioc_context:
166         return None
167
168     ioc_value = ioc_context.get("ioc_value", "unknown")
169
170     direct_indicators = ioc_context.get("direct_indicators") or []

```

```

171 pattern_matches = ioc_context.get("pattern_matches") or []
172
173 all_indicators = direct_indicators + pattern_matches
174 unique_indicators = _deduplicate_indicators(all_indicators)
175
176 lines = [
177     "### IOC ANALYZED",
178     f"- Value: {ioc_value}",
179     f"- Direct pattern indicators found: {len(direct_indicators)}"
180 ,
181     f"- Pattern-based indicators found: {len(pattern_matches)}",
182     f"- Unique indicators shown: {len(unique_indicators)}",
183     "",
184     "### INDICATORS ASSOCIATED WITH THIS IOC",
185     "The following indicators are EXPLICITLY RELATED to this IOC",
186     "according to OpenCTI.",
187     ""
188 ]
189
190 if unique_indicators:
191     for indicator in unique_indicators:
192         lines.append(_render_indicator(indicator))
193     lines.append("")
194 else:
195     lines.append("- No explicit indicators found.")
196     lines.append("")
197
198 related_reports = ioc_context.get("related_reports") or []
199 if related_reports:
200     lines.append("### RELATED REPORTS")
201     for report in related_reports:
202         name = report.get("name") or report.get("description") or
203         report.get("id")
204         lines.append(f"- {name}")
205
206 return "\n".join(lines).strip()

```

Listado 15: ioc_context_builder.py

```

1 # context_profiles.py
2 CONTEXT_PROFILES = {
3     "small": {
4         "max_contextual_docs": 2,
5         "contextual_detail": "compact",
6     },
7     "large": {
8         "max_contextual_docs": 8,
9         "contextual_detail": "full",
10    },
11 }
12
13 def get_context_profile(name: str):
14     return CONTEXT_PROFILES.get(
15         name,
16         CONTEXT_PROFILES["small"]
17     )

```

Listado 16: context_profiles.py

```

1 # builder.py
2 from typing import List
3 from rag.context.context_profiles import get_context_profile
4
5 # Si el contexto es grande y hay que recortar el contexto inferido,
6 # sacar información relevante en vez de un identificador ilegible.
7 def _extract_human_header(doc: str) -> str:
8     lines = [l.strip() for l in doc.splitlines() if l.strip()]
9
10    for line in lines:
11        if ":" in line and not line.startswith("["):
12            return line
13
14    for line in lines:
15        if not (line.startswith("[") and line.endswith("]")):
16            return line
17
18 def _extract_section_items(lines: List[str], section_names: List[str])
19     -> List[str]:
20     """
21     Extrae elementos tipo lista bajo secciones como:
22     - TTPs usados:
23     - Malware asociado:
24     - Threat actors asociados:
25     """
26
27     items = []
28     capture = False
29
30     normalized_sections = [s.lower() for s in section_names]
31
32     for line in lines:
33         lower = line.lower().strip()
34
35         if any(lower.startswith(section) for section in
36             normalized_sections):
37             capture = True
38             continue
39
40         if capture and lower.endswith(":") and not line.startswith("-"
41             ):
42             break
43
44         if capture:
45             if line.startswith("-"):
46                 value = line[1:].strip()
47                 if value and value.lower() not in ("n/a", "none", "
48                 null"):
49                     items.append(value)
50             elif line and not line.startswith("-"):
51                 break
52
53     return items
54
55 def _extract_first_matching_line(lines: List[str], prefixes: List[str
56     ]) -> str | None:
57     normalized_prefixes = [p.lower() for p in prefixes]

```

```

53
54     for line in lines:
55         lower = line.lower().strip()
56         if any(lower.startswith(prefix) for prefix in
normalized_prefixes):
57             return line
58
59     return None
60
61
62 def _extract_compact_context(doc: str) -> str:
63     lines = [l.strip() for l in doc.splitlines() if l.strip()]
64
65     if not lines:
66         return "- CTI_OBJECT (empty document)"
67
68     header = _extract_human_header(doc)
69
70     description = _extract_first_matching_line(
71         lines,
72         [
73             "descripción:",
74             "descripcion:",
75             "description:",
76         ]
77     )
78
79     created = _extract_first_matching_line(
80         lines,
81         [
82             "creado:",
83             "created:",
84         ]
85     )
86
87     modified = _extract_first_matching_line(
88         lines,
89         [
90             "modificado:",
91             "modified:",
92             "updated",
93         ]
94     )
95
96     ttps = _extract_section_items(
97         lines,
98         [
99             "modificado:",
100             "modified:",
101             "updated:",
102         ]
103     )
104
105     ttps = _extract_section_items(
106         lines,
107         [
108             "ttps usdos:",
109             "ttps:",

```

```

110         "techniques:",
111         "attack_patterns:",
112         "attack_patterns:",
113     ]
114 )
115
116 malware = _extract_section_items(
117     lines,
118     [
119         "malware asociado:",
120         "associated malware:",
121         "malware:",
122     ]
123 )
124
125 threat_actors = _extract_section_items(
126     lines,
127     [
128         "threat actors asociados:",
129         "associated threat actors:",
130         "threat actors:",
131         "actors:",
132     ]
133 )
134
135 reports = _extract_section_items(
136     lines,
137     [
138         "reports asociados:",
139         "associated reports",
140         "reports",
141     ]
142 )
143
144 indicators = _extract_section_items(
145     lines,
146     [
147         "indicators asociados:",
148         "indicadores asociados:",
149         "associated indicators:",
150         "indicators:",
151         "indicadores:",
152     ]
153 )
154
155 output = [f" - {header}"]
156
157 if description:
158     output.append(f"         {description}")
159
160 if ttps:
161     output.append(" TTPs / Techniques:")
162     for item in ttps[:5]:
163         output.append(f"         - {item}")
164
165 if malware:
166     output.append(" Associated malware:")
167     for item in malware[:5]:

```



```

168         output.append(f"        - {item}")
169
170     if threat_actors:
171         output.append(" Associated threat actors:")
172         for item in threat_actors[:5]:
173             output.append(f"        - {item}")
174
175     if reports:
176         output.append(" Associated reports:")
177         for item in reports[:3]:
178             output.append(f"        - {item}")
179
180     if indicators:
181         output.append(" Associated indicators:")
182         for item in indicators[:5]:
183             output.append(f"        - {item}")
184
185     if created:
186         output.append(f"        {created}")
187
188     if modified:
189         output.append(f"        {modified}")
190
191     return "\n".join(output)
192
193
194 def _prepare_contextual_docs(context_docs: List[str], detail: str) ->
List[str]:
195     if detail == "titles":
196         return [_extract_human_header(doc) for doc in context_docs]
197
198     if detail == "compact":
199         return [_extract_compact_context(doc) for doc in
context_docs]
200
201     # Si el nivel de detalle es completo
202     return context_docs
203
204 def build_final_context(
205     *,
206     ioc_context_text: str | None,
207     context_docs: List[str],
208     context_profile: str,
209 ) -> str:
210     profile = get_context_profile(context_profile)
211
212     context_blocks = []
213     # CONTEXTO FUERTE
214     if ioc_context_text:
215         context_blocks.append(
216             f"{ioc_context_text}"
217         )
218
219     # CONTEXTO DÉBIL
220     if context_docs:
221         docs = context_docs[:profile["max_contextual_docs"]]
222         docs = _prepare_contextual_docs(docs, profile["
contextual_detail"])

```

```
223
224
225     context_blocks.append(
226         "### CONTEXTUAL CTI INFORMATION (INFERRED FROM SIMILARITY)
\n"
227         "The following CTI-related information is retrieved via
semantic "
228         "similarity search.\n"
229         + "\n".join(docs)
230     )
231
232     return "\n\n".join(context_blocks)
```

Listado 17: builder.py

Bibliografía

- [1] Davidjbianco. *Enterprise Detection & Response: The Pyramid of Pain*. Enterprise Detection & Response. 1 de mar. de 2013. URL: <https://detect-respond.blogspot.com/2013/03/the-pyramid-of-pain.html> (visitado 27-05-2026).
- [2] OpenCTI Team. *OpenCTI Data Model*. OpenCTI Data Model. URL: <https://docs.opencti.io/latest/usage/data-model/> (visitado 27-05-2026).
- [3] *Modelo extenso de lenguaje*. En: *Wikipedia, la enciclopedia libre*. 25 de mayo de 2026. URL: https://es.wikipedia.org/w/index.php?title=Modelo_extenso_de_lenguaje&oldid=173636604 (visitado 27-05-2026).
- [4] Cole Stryker. *¿Qué son los grandes modelos de lenguaje (LLM)?* — IBM. 6 de oct. de 2021. URL: <https://www.ibm.com/es-es/think/topics/large-language-models> (visitado 27-05-2026).
- [5] *¿Qué es un LLM? - Explicación de los modelos de lenguaje grandes - AWS*. Amazon Web Services, Inc. URL: <https://aws.amazon.com/es/what-is/large-language-model/> (visitado 27-05-2026).
- [6] Woosuk Kwon et al. «Efficient Memory Management for Large Language Model Serving with PagedAttention». En: *Proceedings of the 29th Symposium on Operating Systems Principles*. SOSP '23: 29th Symposium on Operating Systems Principles. Koblenz Germany: ACM, 23 de oct. de 2023, págs. 611-626. ISBN: 979-8-4007-0229-7. DOI: 10.1145/3600006.3613165. URL: <https://dl.acm.org/doi/10.1145/3600006.3613165> (visitado 29-05-2026).
- [7] *Segolilylabs/Lily-Cybersecurity-7B-v0.2 · Hugging Face*. URL: <https://huggingface.co/segolilylabs/Lily-Cybersecurity-7B-v0.2> (visitado 29-05-2026).
- [8] Matan Levi et al. *Toward Cybersecurity-Expert Small Language Models*. Ver. 1. 2025. DOI: 10.48550/ARXIV.2510.14113. URL: <https://arxiv.org/abs/2510.14113> (visitado 29-05-2026). Prepublicado.
- [9] OpenWebUI. *Open WebUI*. URL: <https://docs.openwebui.com/> (visitado 01-06-2026).
- [10] OpenWebUI. *Pipelines / Open WebUI*. URL: <https://openwebui.com/features/extensibility/pipelines/> (visitado 01-06-2026).
- [11] MITRE. *MITRE ATT&CK®*. URL: <https://attack.mitre.org/> (visitado 08-06-2026).